

생성형 모델 기반 로봇 행동 제어

Index



01

DR4R의 소개

02

정규분포에서 확산모델로

03

ACT

04

Diffusion Policy

05

Consistency Policy

01 DR4R 소개

모두의연구소 RL4RWS - Reinforcement Learning for Real World System

- 2017년 9월 시작

모두의연구소 DR4R - Deep learning Reinforcement Learning for Real World

- 2022년 8월 시작

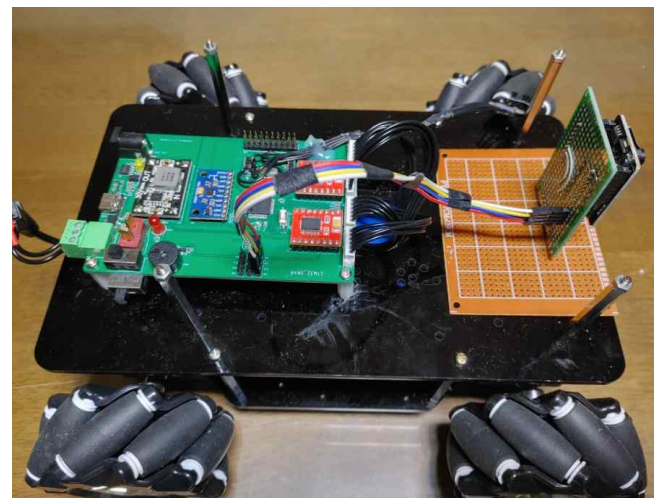
DR4R

- 시즌 1 2022년 08월 13일
- 시즌 2 2023년 01월 07일
- 시즌 3 2023년 07월 15일
- 시즌 4 2023년 12월 02일
- 시즌 5 2024년 05월 11일
- 시즌 6 2025년 01월 11일
- 시즌 7 2025년 06월 21일

ROS GAZEBO

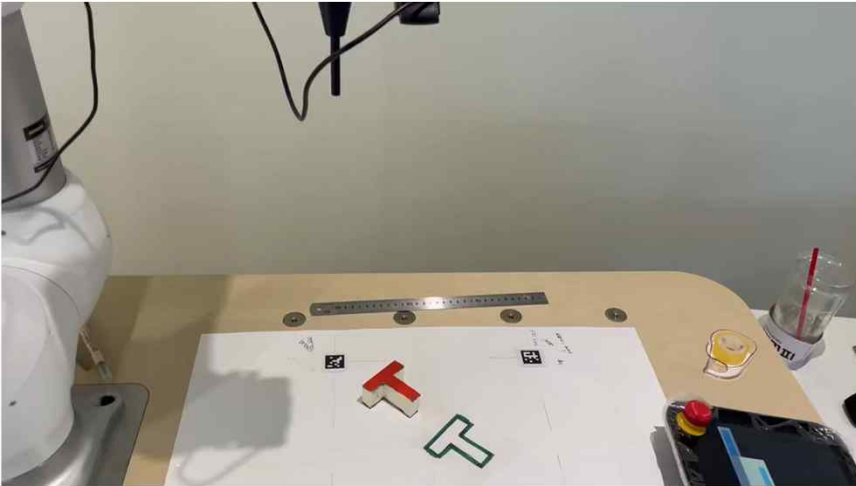


MoveIt

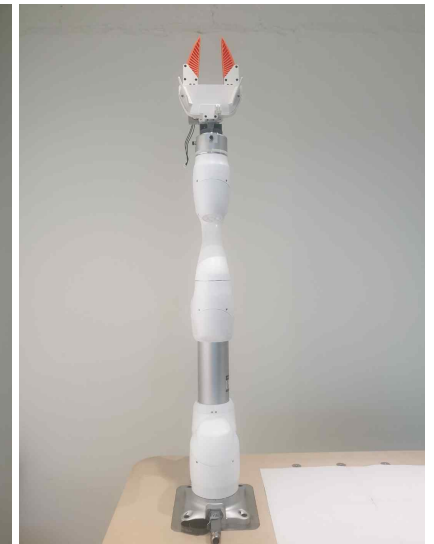


DR4R Season 5

- ✓ 디자인 및 기구설계, HW 분야, 임베디드 시스템 그리고 모방학습 스터디 진행
 - 1. UMI 논문 스터디 및 구현된 내용 분석
 - 2. 모방학습 스터디 및 시뮬레이터 구현
 - 3. 3D 프린터로 모방학습용 gripper를 제작
 - 4. 저가의 로봇팔 제작



[PushT 실행 영상 링크](#)



Diffusion Policy & UMI site

https://github.com/real-stanford/universal_manipulation_interface

https://github.com/real-stanford/diffusion_policy

DR4R Season 6, 7

✓ DR4R Season6

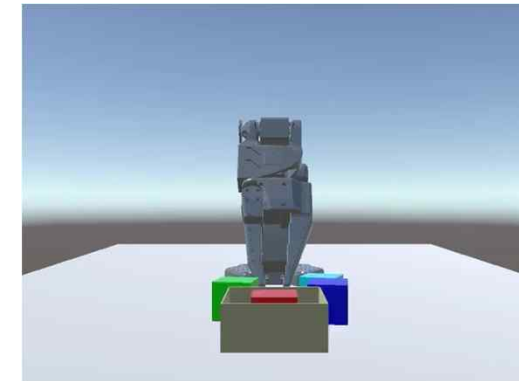
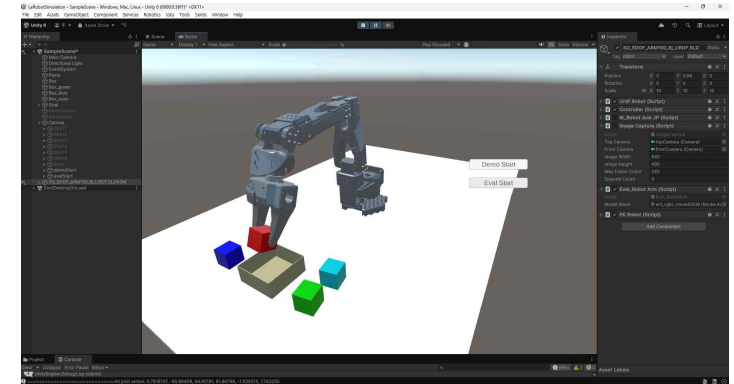
□ LeRobot 선정 배경

- 저가의 로봇
- 다양한 모델이 하나의 시스템에 구현됨
- 자체 로봇 플랫폼 구축

□ 진행내용

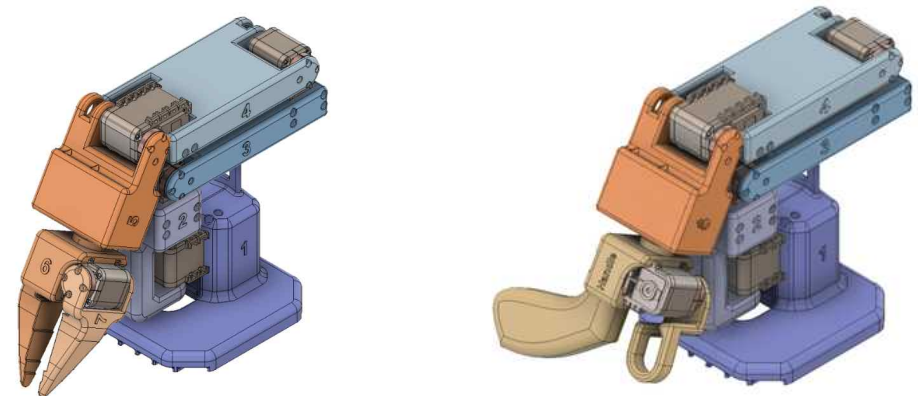
- ACT, Diffusion Policy, Pi0 모델 스텀디
- Lerobot 자체 버전 설계
- Unity Simulation 구현

UNITY Simulation



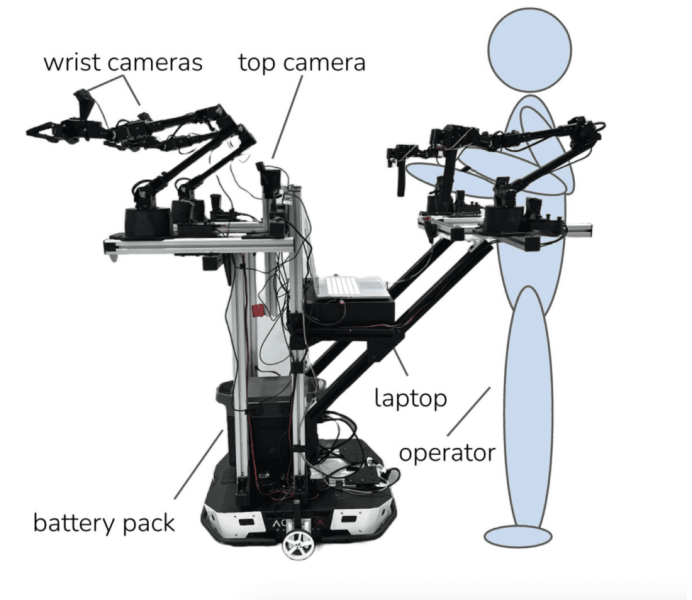
✓ DR4R Season7

- ACT, diffusion policy, Pi0, SmoIVLA 동작 테스트
- Consistency model 완성
- Unity simulator 고도화
- 실용적인 Application 구현



LeRobot 디자인 및 기구설계 – DR4R version

Mobile Aloha – ACT – VAE



- 제작비 5천만원 이상 고가
- 모든 시스템이 동일한 사양이어야 함

Universal Manipulation Interface (스탠포드)

Diffusion Policy - DDPM



Consistency Policy

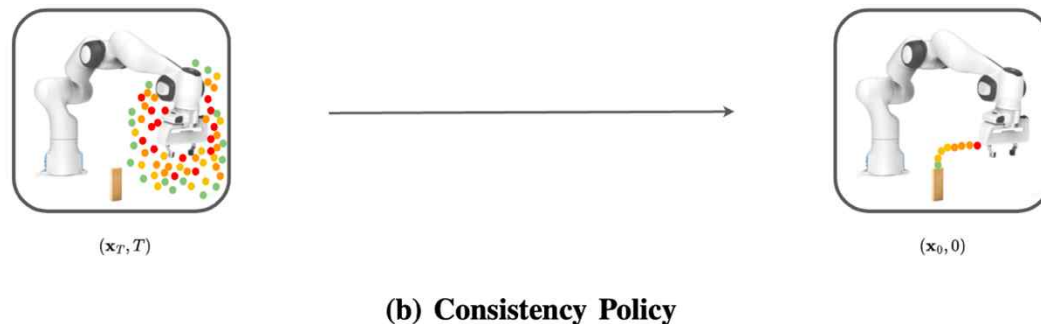
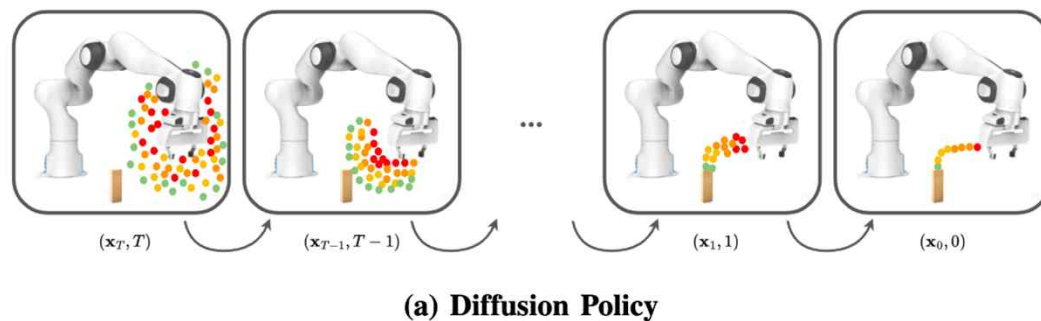
✓ Score function

- 시각 기반 로봇 제어 학습을 위한 Diffusion Policy보다 빠르고 유사한 성능을 지닌 대안으로 Consistency Policy 사용.

- **Consistency Policy**는 **Diffusion Policy**의 추론 속도를 **10배** 향상시키면서도 경쟁력 있는 성능을 유지하는 로봇 제어 정책임.

- **Diffusion Policy**의 궤적을 따라 자기 일관성을 학습하여 증류되며, 노트북 **GPU** 같은 저자원 환경에서도 작동함

- 학습은 사전 정책의 품질에 덜 민감하고, 일관성 목적, 낮은 샘플 분산, 고정 체인 스텝이 성능 향상 핵심임



02 정규분포에서 확산모델로 이르는 10단계 여정

생성형 AI를 위한 확산모델 스터디

모두의연구소

O'REILLY

밑바닥부터
시작하는
딥러닝 5

Deep Learning from Scratch 5



한빛미디어

10단계로 익히는
이미지 생성 모델의 원리

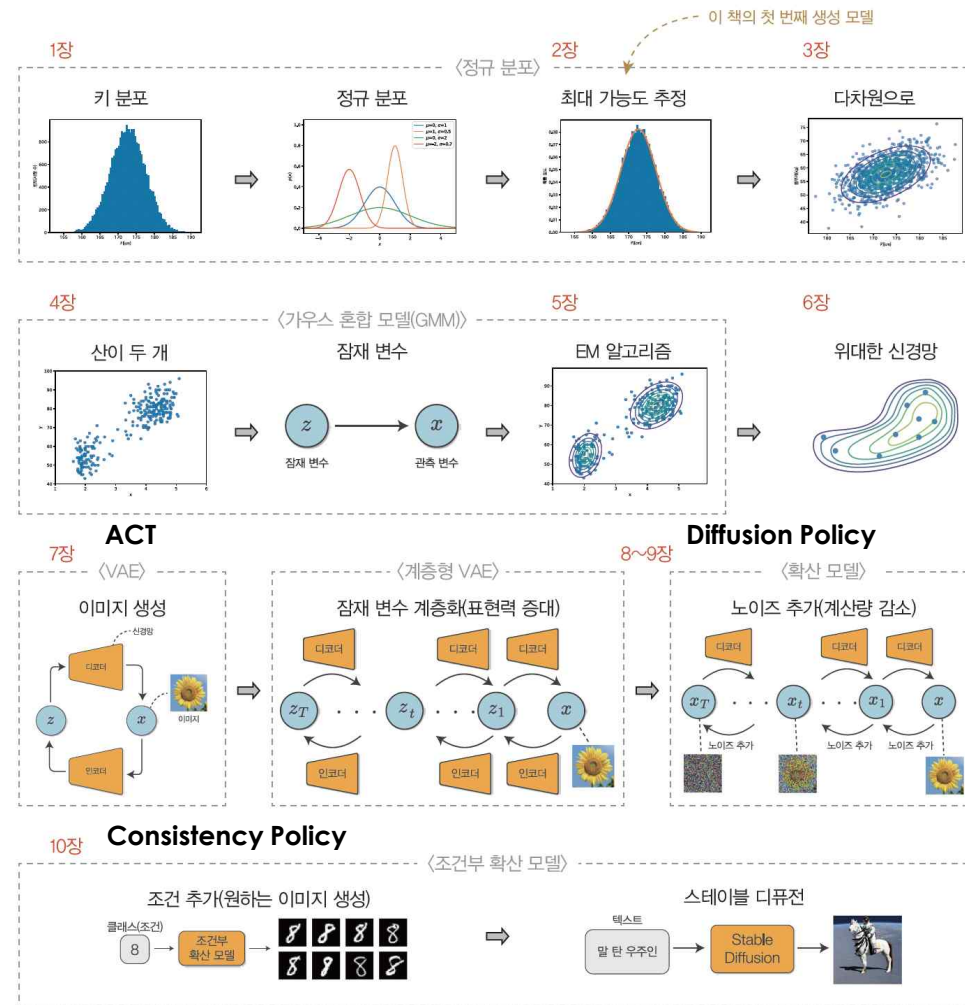
4차원 공간 지각
개념에서 용단

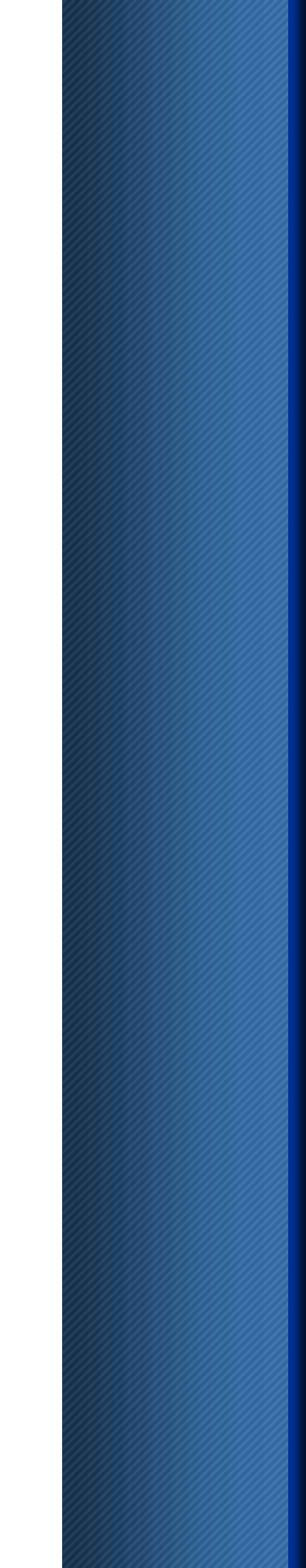
확산 모델에 대한 이론 스터디

- ACT : CVAE

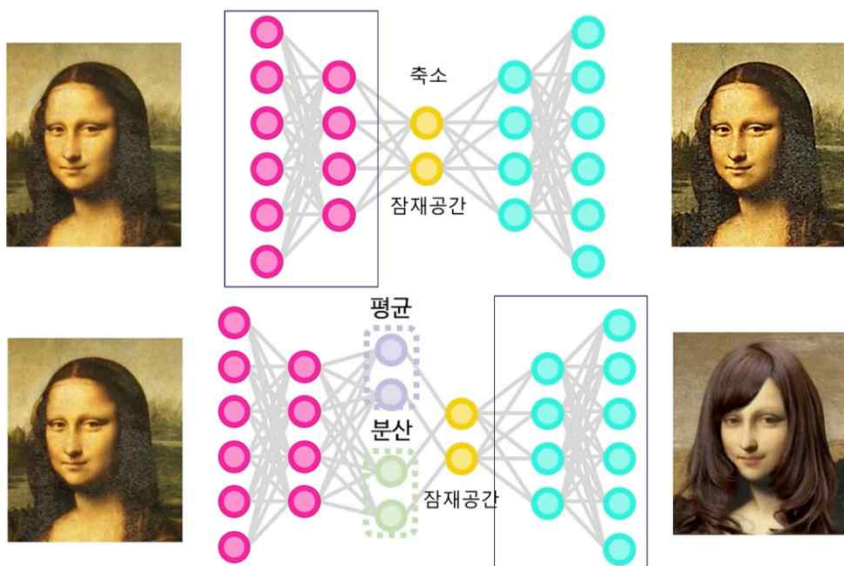
- **D**iffusion Policy : DDPM, DDIM

- Consistency Policy : Score function





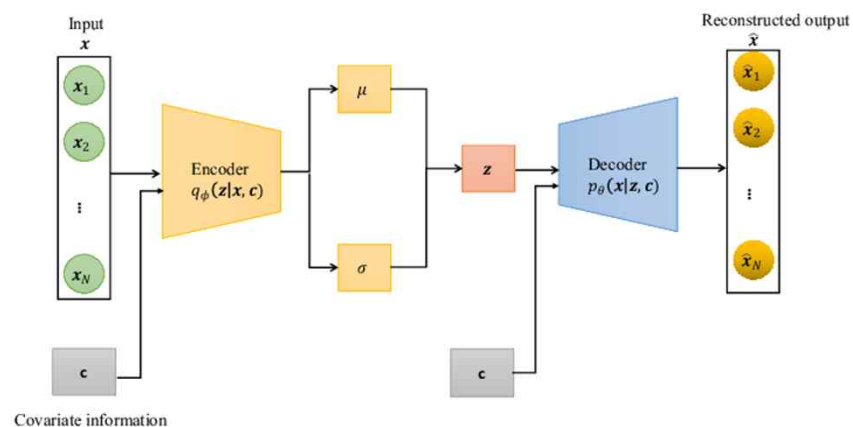
03 **ACT**



AutoEncoder(AE) vs. Variational AutoEncoder(VAE) 차이점

구분	AutoEncoder (AE)	Variational AutoEncoder (VAE)
목적	데이터 압축 및 특징 추출	확률적 잠재 공간 학습 및 생성 모델
잠재 공간	고정된 잠재 벡터 (z)	확률 분포 ($z \sim \mathcal{N}(\mu, \sigma)$)
디코더 입력	고정된 벡터 사용	샘플링된 잠재 변수 사용
출력	입력과 유사한 복원 이미지	입력과 유사하면서도 새로운 데이터 생성 가능
손실 함수	재구성 오차 (L_{recon})	재구성 오차 (L_{recon}) + KL Divergence (L_{KL})

AE는 단순히 데이터를 압축하고 복원하는 역할을 하지만, VAE는 확률 분포를 학습하여 새로운 데이터를 생성할 수 있습니다.



- **Variational Autoencoder(VAE)**는 입력 데이터를 잠재공간(latent space)에 확률 분포로 인코딩하고, 이를 샘플링하여 복원하는 생성 모델임.
- 재구성과 잠재분포의 정규성 간 균형을 맞추기 위해 **ELBO**(증분가능한 하한)를 최적화함.
- **VAE**는 이미지 생성, 이상 탐지 등 다양한 생성적 작업에 활용됨.

Transformer

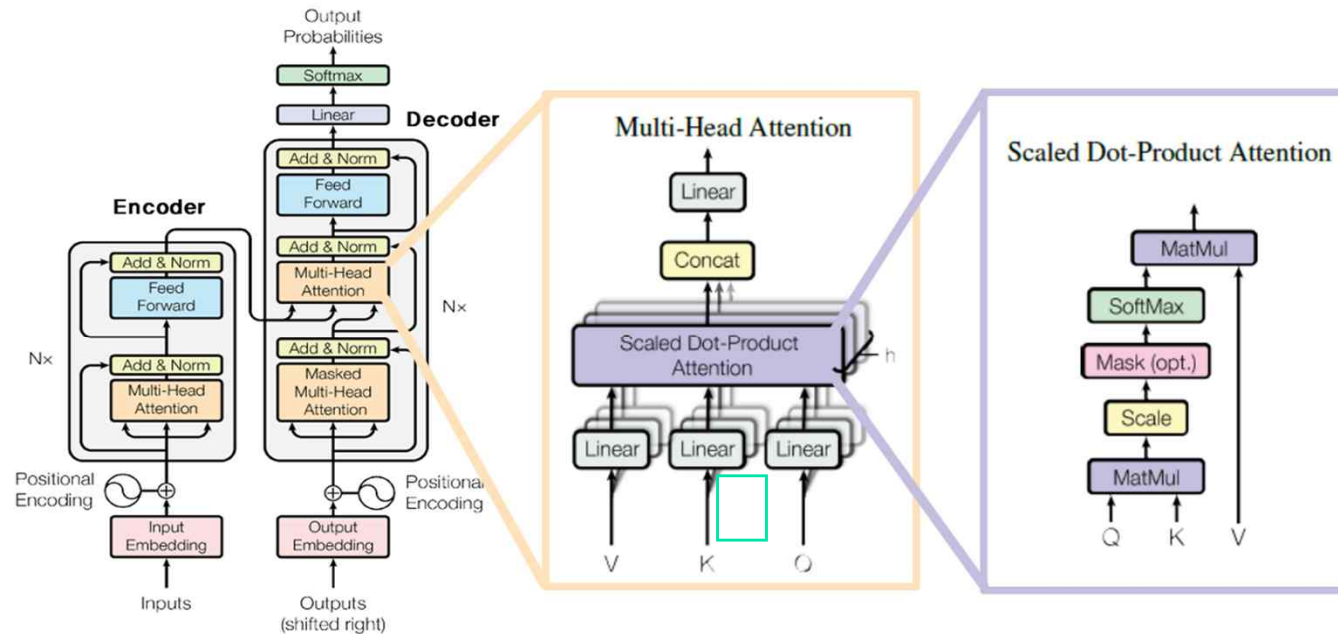


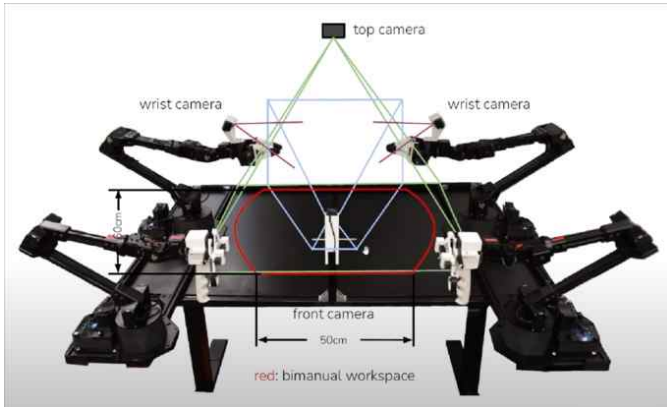
Figure 1: The Transformer - model architecture.

Transformer의 주요 학습 대상 요약

구성 요소	학습 여부	역할
Embedding Layer	✓ 학습함	입력 토큰을 벡터로 변환
Query / Key / Value Projection (Linear)	✓ 학습함	주의(attention) 대상과 범위 조정
Attention Weight 계산	✓ 간접 학습	$\text{softmax}(QK^T / \sqrt{d})$ 로 동적 가중치 결정
FeedForward Layer	✓ 학습함	비선형 표현 확장
LayerNorm, Dropout 등	✓ 학습함 (일부)	안정화 및 regularization
Positional Embedding	✓ (학습형일 경우)	순서 정보 반영
CLS Token 등 특수 임베딩	✓ 학습함	전체 시퀀스 표현 등

ACT 개요

MobileALOHA 4개의 카메라

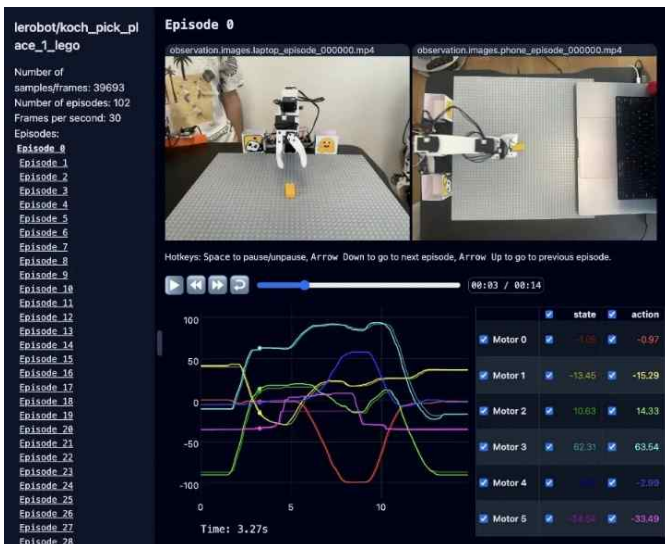


인간 시연(행동 복제)

human demonstrations (behavior cloning)

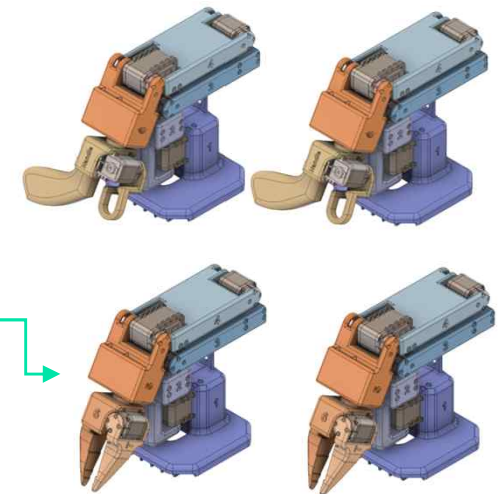
- 데이터 수집 : 인간 시연(human demonstrations)을 통해
- 학습 : 행동 복제(behavior cloning)
- 테스트 : 복제 행동 실행

LeRobot 2개의 카메라



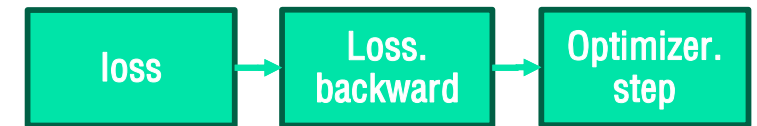
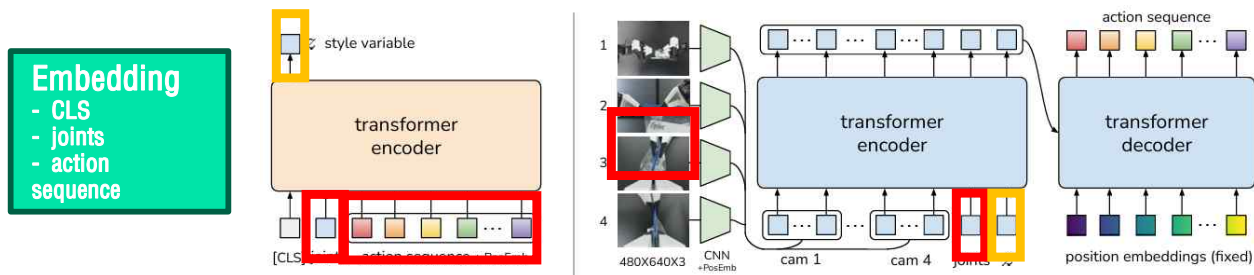
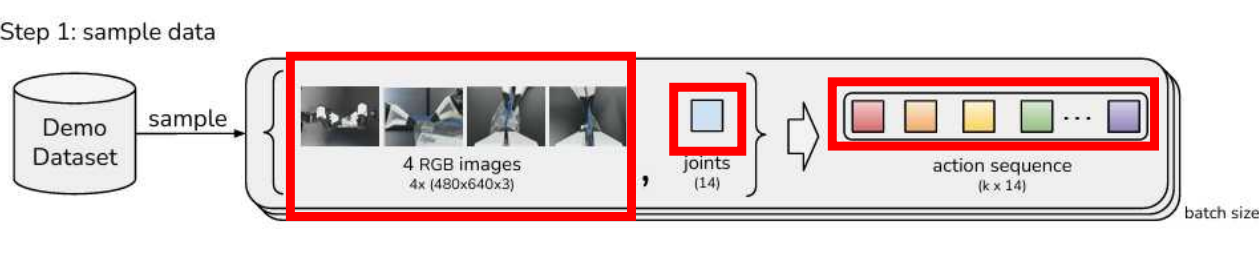
ACT

스타일 변수 z :
로봇 행동의 규칙 학습

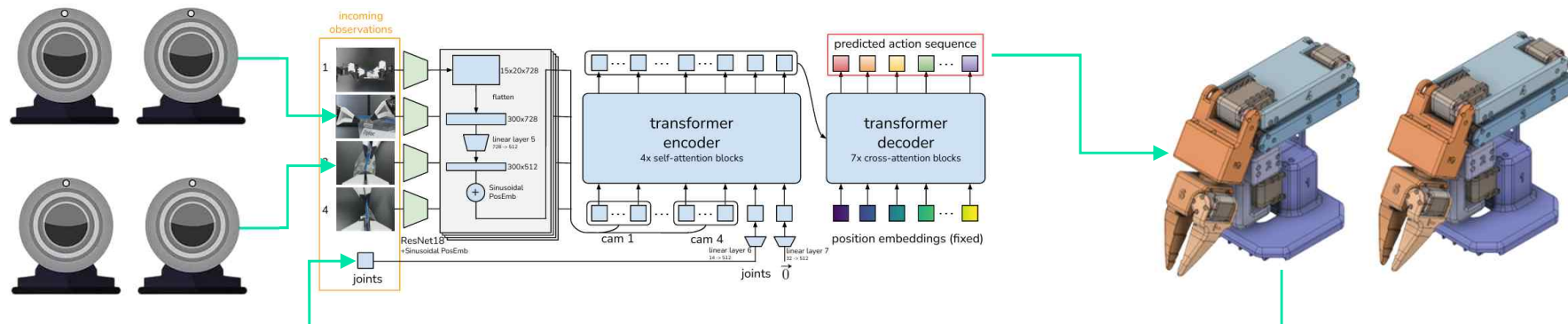


ACT 전체 아키텍처

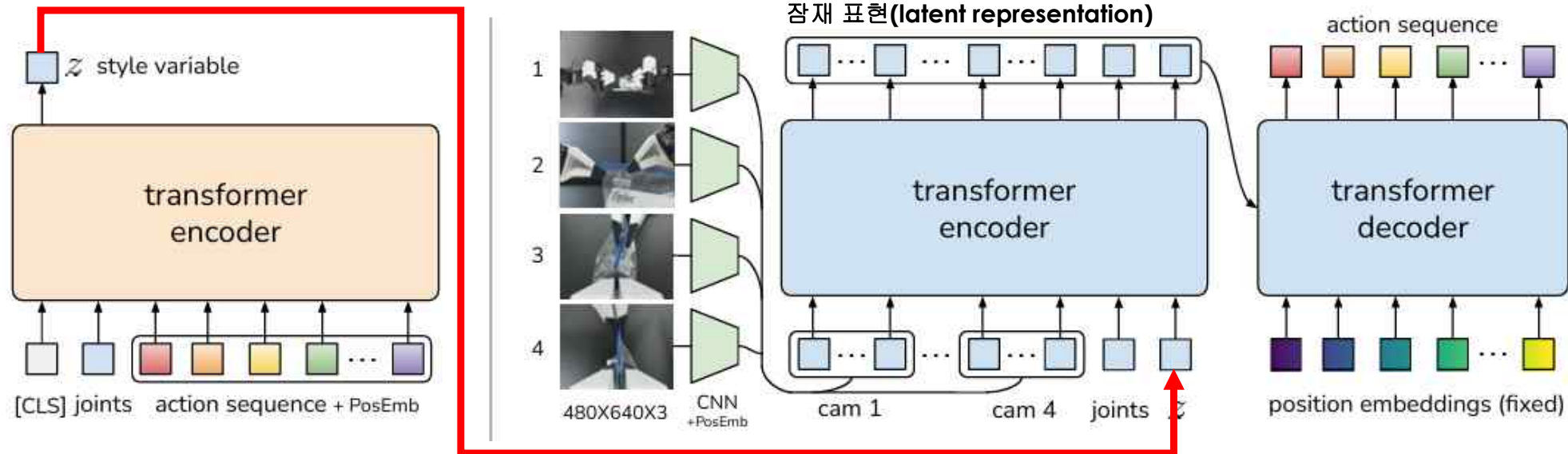
Step 1: sample data



$$\mathcal{L}_{VAE} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}[q(z|x) \parallel p(z)]$$



ACT 전체 아키텍처

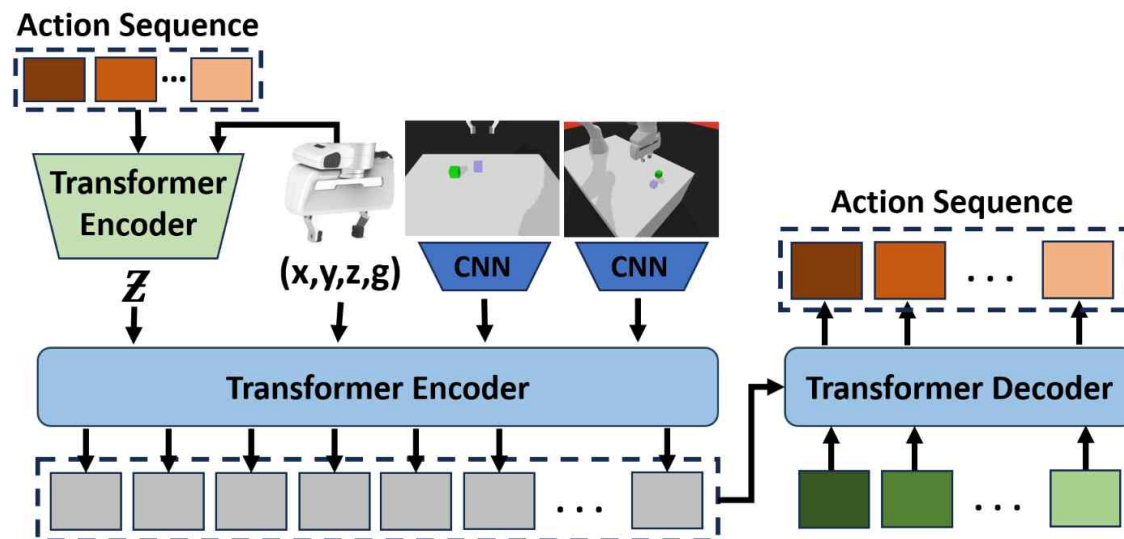


왼쪽: CVAE의 인코더

- action sequence와 joints observation 압축하여 스타일 변수 z 로 만듦.
- 인코더 : 테스트 시 사용되지 않음

오른쪽: ACT의 디코더

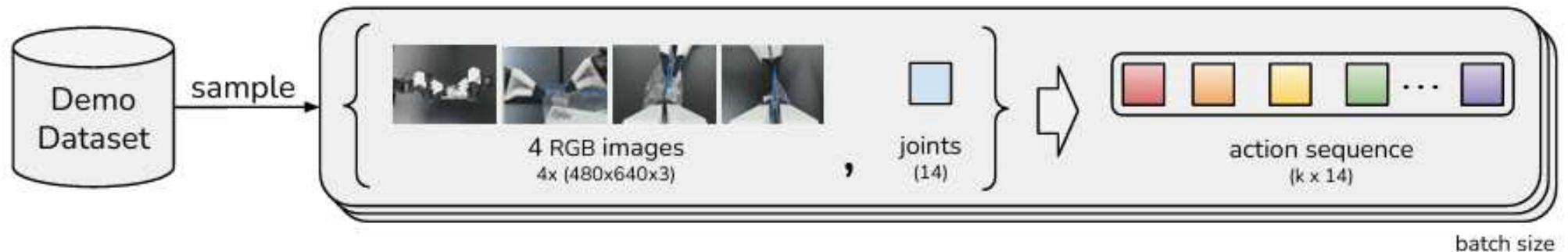
- 여러 시점에서의 이미지, 관절 위치 정보, z 를 **Transformer** 인코더에 입력해 처리
- **Transformer** 디코더 통해 행동 시퀀스 예측
- 테스트 시에는 z 는 prior의 평균값, 즉 **0**으로 설정



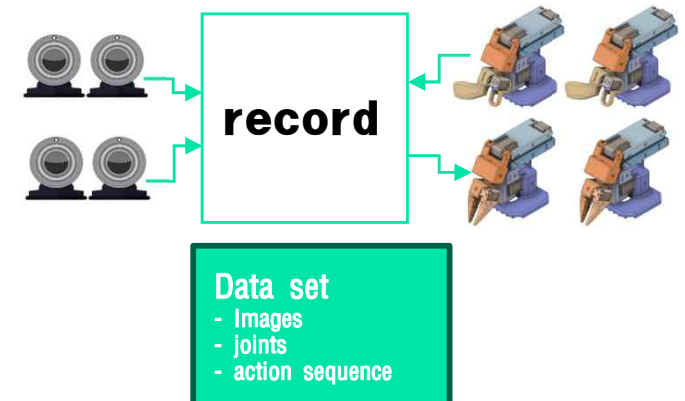
1. Training – Demo Data Set 얻기

1. 이미지 (480 x 640 x 3)와 관절 위치 데이터(1 x 14) 얻기

Step 1: sample data

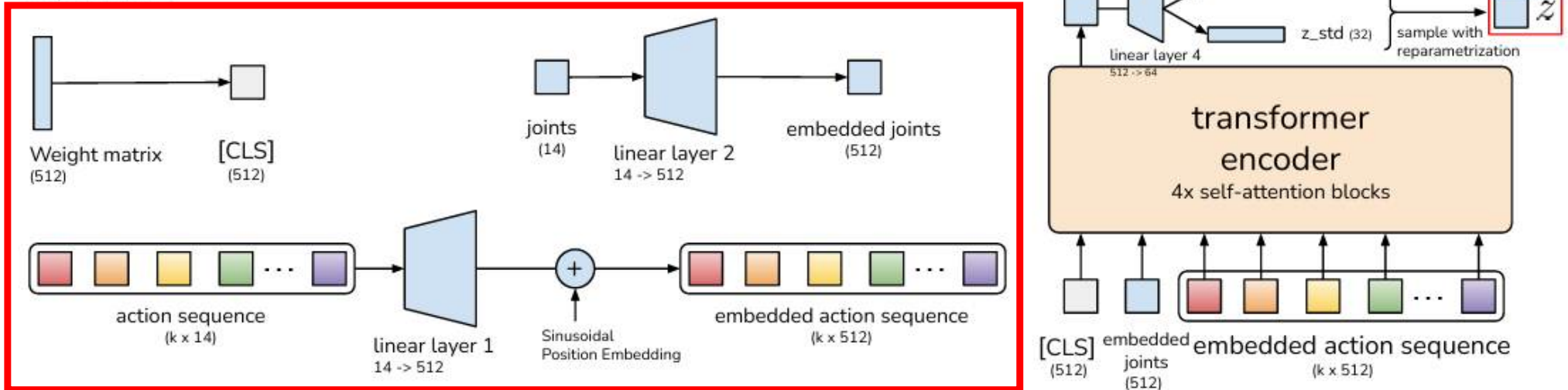


- 데이터셋은 이미지(480x640x3)와 로봇의 관절 위치 데이터(1x14) 포함
- 관절 위치 데이터, Joints : 1x14 (로봇의 관절 6개 + 그리퍼 1개) x 두 개의 로봇(좌/우 팔)
 - 14개 = 6(왼쪽 관절) + 1(왼쪽 그리퍼) + 6(오른쪽 관절) + 1(오른쪽 그리퍼)
- k개의 타임스텝으로 쌓아서 (k x 14) 형태의 액션 시퀀스를 만듦



Training

Step 2: infer z



2. 관절 위치 데이터 1×512 로 변환

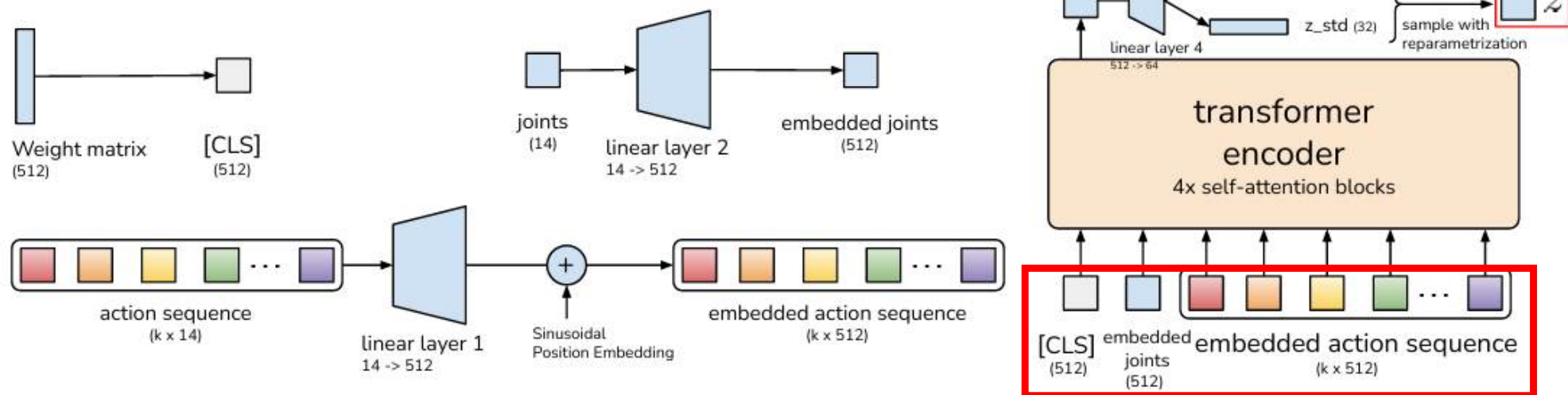
- $1 \times 14 \rightarrow 1 \times 512$ 로 변환
- 512차원의 임베딩 공간으로 변환하는 과정입니다.
- 선형층(**Linear Layer**) 사용하여 **Transformer** 모델과 데이터 크기를 맞춤

3. 액션 시퀀스 ($k \times 512$)로 변환하여 사인파 위치 포함

- 액션 시퀀스 ($k \times 14$) $\rightarrow k \times 512$ 로 변환
- 14차원 관절 데이터를 **Transformer**가 다룰 수 있도록 선형층을 사용하여 ($k \times 512$) 차원으로 변환
- 사인파 위치 포함(**Positional Encoding**)
 - **Transformer**는 순서를 모르는 모델이기 때문에, 시퀀스 정보를 유지하기 위해 사인파 위치 임베딩 추가

Training

Step 2: infer z

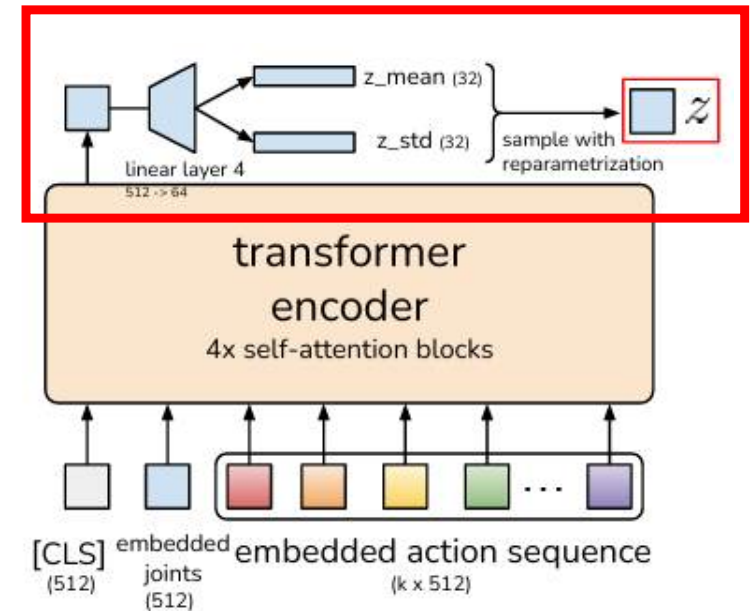
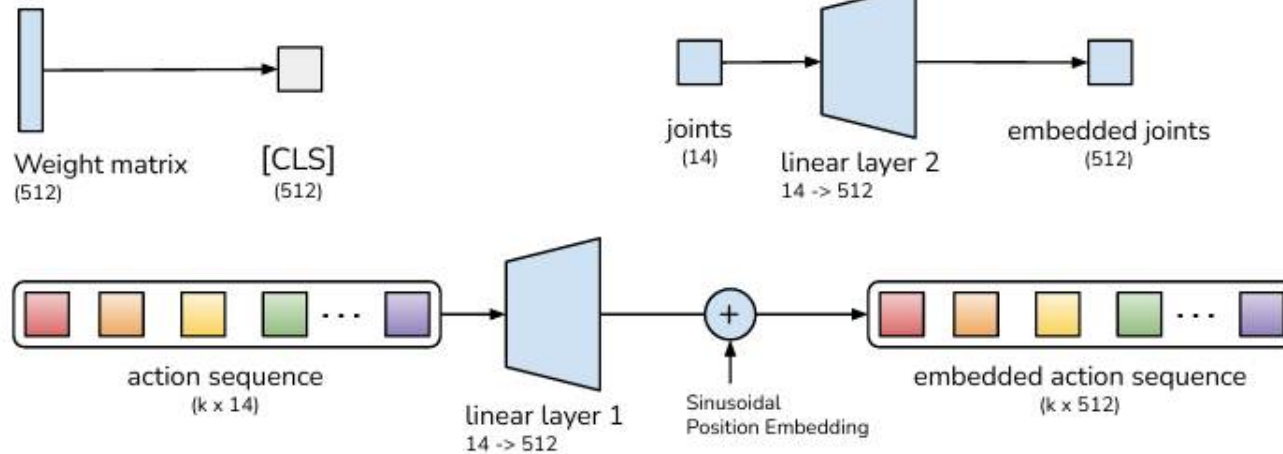


4. 관절 위치 데이터, 액션 시퀀스 및 [CLS]를 4개의 **Self-Attention** 블록이 있는 트랜스포머 인코더에 전달

- **Transformer Encoder**에 들어가는 입력 : [관절 위치 데이터, 액션 시퀀스, **CLS**]
- **CLS** : "**C**lassification **T**oken"의 약자
 - 전체 시퀀스를 요약하는 대표 벡터를 생성하는 특별한 토큰입니다.

Training

Step 2: infer z



5. Transformer의 출력을 선형 층을 통해 평균과 분산 획득

- Transformer Encoder의 최종 출력에서 평균 (1x32)와 분산 (1x32)를 구함.
- Variational Autoencoder(VAE)에서 수행되는 과정과 동일

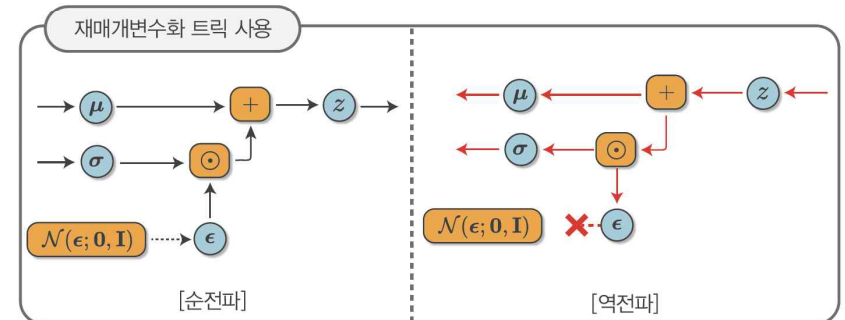
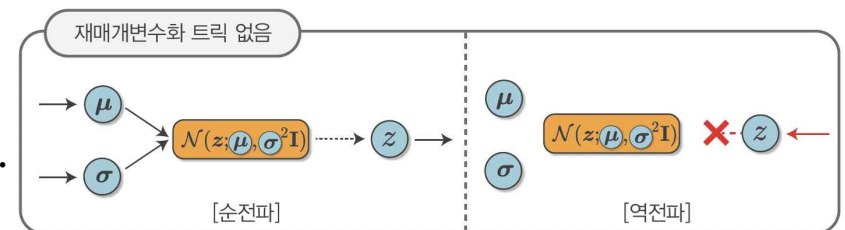
6. 재 매개 변수화 트릭을 사용하여 스타일 변수 z 샘플링

- Reparameterization Trick을 사용하여 잠재 변수 z (1 x 32) 샘플링

$$z = u + \sigma \cdot \epsilon, \text{ where } \epsilon \sim N(0, 1)$$

- 이 과정이 있어야 VAE가 미분 가능해지고, 모델이 학습할 수 있습니다.

재매개변수화 트릭에 의해 달라지는 계산 그래프(역전파의 빨간 화살표는 기울기 전파를 나타냄)



ACT에서 잠재 변수 z 의 의미해석

latent 변수 z 의 물리적 의미

CVAE에서의 latent 변수는 행동 chunk의 잠재적 형태나 스타일, 혹은 **고수준 의도(high-level intent)**를 추상적으로 나타냅니다. 좀 더 구체적으로 설명하면:

Latent z 의 물리적 의미

- 행동의 종류: 같은 목표라도 "빠르게 접근하기" vs "조심스럽게 접근하기"처럼 다양한 전략을 나타낼 수 있습니다.
- 모션 패턴: 예를 들어, 같은 타겟을 향한 이동에서도 곡선 경로 vs 직선 경로 등.
- 고차원 행동 표현의 압축: 길이가 긴 action chunk를 압축한 표현 (chunk-level representation)

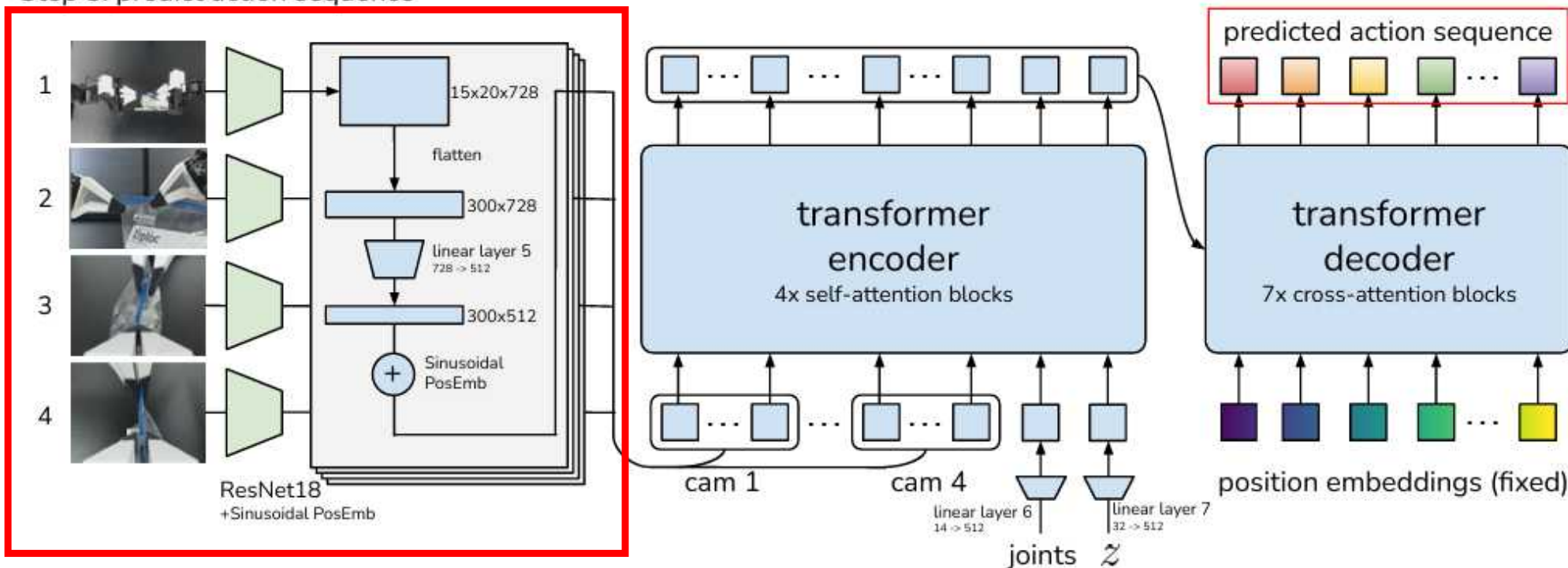
즉, z 는 행동 시퀀스의 추상적인 요약값이며, 다양한 행동 분포를 생성 가능하게 해주는 핵심 요소입니다.

Latent vector (dim=32)의 잠재적 의미 예시

차원	잠재적 의미 (예시)	설명
0~1	움직임 속도	빠른 행동 vs 느린 접근
2~3	경로 형태	직선형 이동 vs 곡선 우회
4~5	목표 집중도	목표 중심 vs 환경 대응 위주
6~7	움직임의 강도	세게, 빠르게 vs 조심스럽게
8~9	좌우 중심성	왼쪽 공간 우선 vs 오른쪽 우선
10~11	시작 포즈 변화량	시작자세가 바뀔 수 있는 여유도
12~15	회피 성향	장애물 회피 vs 무시
16~20	행동 스타일	보폭 크기, 움직임 패턴 등
21~25	환경 반응 민감도	주변 환경 변화에 즉각 반응하는 정도
26~28	불확실성 표현	랜덤성, 행동 다양성 보장 정도
29~31	목표 거리 표현	가까운 목표 vs 먼 목표에 적합한 전략

Training

Step 3: predict action sequence

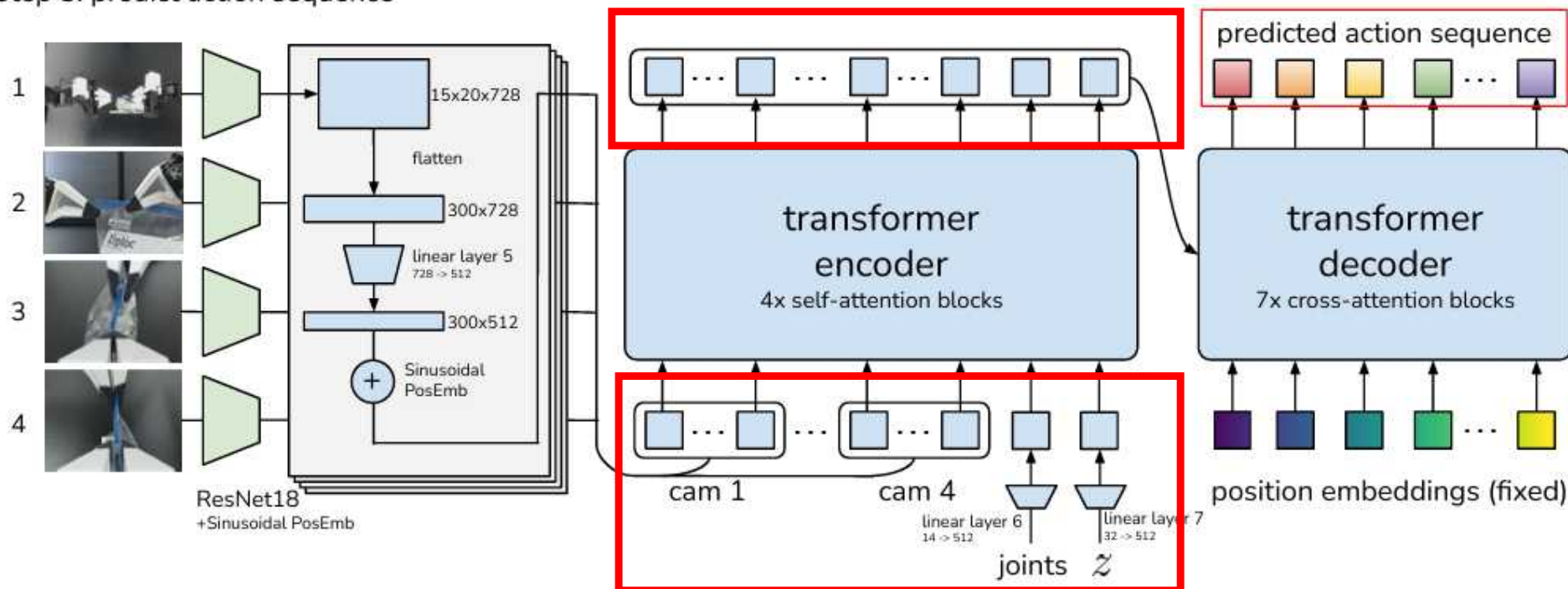


7. 이미지 특징 추출

- ResNet18을 통과한 후 이미지 특징을 변환하는 과정입니다.
- 출력 크기 계산:
 - ResNet18의 최종 출력 : (15 x 20 x 728)
 - (300 x 728) → (300 x 512)로 변환 후 Transformer 입력으로 사용
 - 사인파 위치 임베딩 추가 : Transformer가 시공간 정보를 이해하도록 위치 임베딩 추가

Training

Step 3: predict action sequence



8. [이미지 특징, joints, z] $\rightarrow$ Transformer Encoder 입력

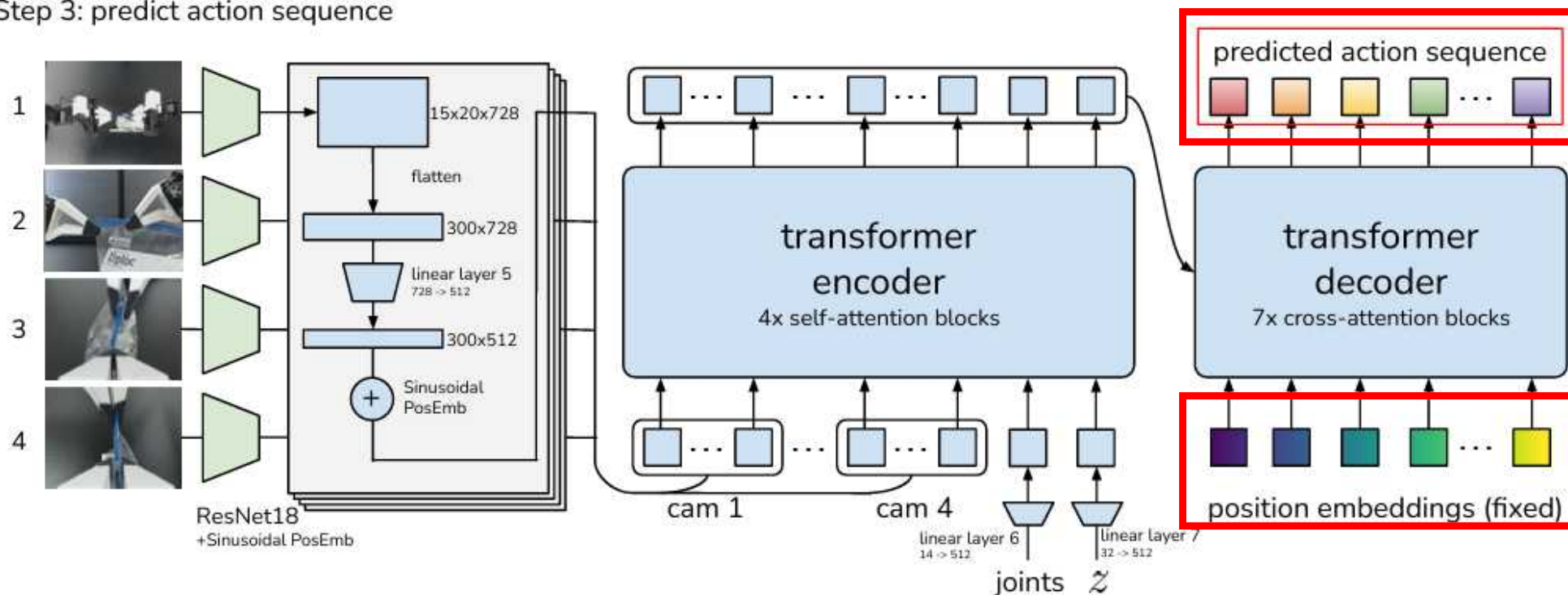
- Transformer Encoder의 입력: (이미지 특징 + 관절 상태 + z)
- 각 입력을 512차원으로 변환 후 Self-Attention을 수행하여 통합된 표현 생성

9. 트랜스포머 디코더 : 트랜스포머 인코더의 출력을 조건으로 동작예측

- 7개의 Cross-Attention 블록을 사용하여 과거 정보와 현재 조건 결합

Training

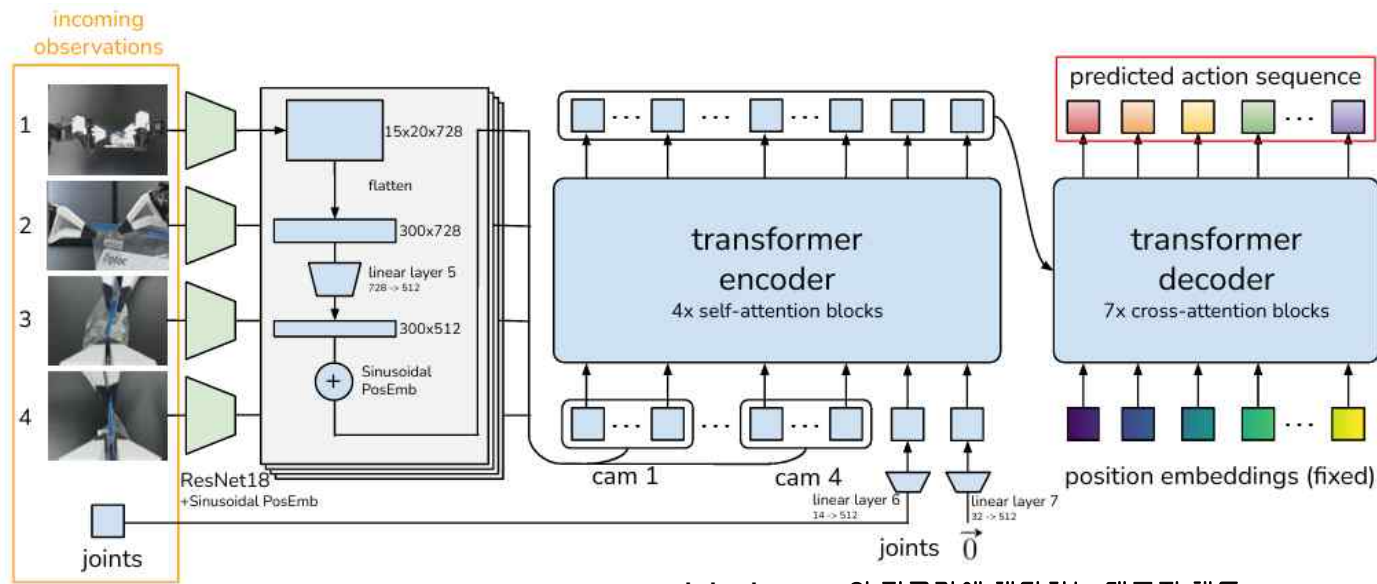
Step 3: predict action sequence



10. 트랜스포머 디코더에 고정값을 입력하여 출력 액션 시퀀스를 얻음

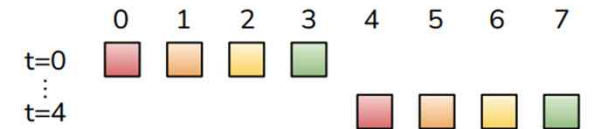
- ($k \times 512$) 형태의 고정 위치 임베딩 벡터를 **Transformer Decoder**에 입력
- **Transformer Decoder**는 예측된 ($k \times 14$) 크기의 액션 시퀀스를 출력합니다.

Testing

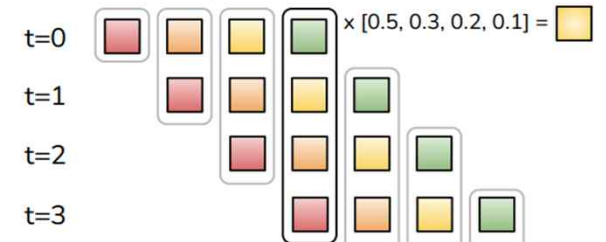


latent space의 평균값에 해당하는 대표적 행동

Action Chunking



Action Chunking + Temporal Ensemble



$z = 0$ 으로 두는 이유

- **stochastic latent (z)**를 고정시키고, 현재 관측(observation)과 로봇 상태만을 기반으로 결정적인 행동 시퀀스(action chunk)를 생성하기 위함

Temporal Ensemble (시간적 앙상블)

- 청크를 하나씩 순차적으로 실행하면, 동작이 매끄럽지 못하고 단절될 가능성이 있음
- 시간축을 따라 중첩된 청크가 존재하므로, 동일한 시간에 대응되는 액션들을 평균화하여 부드러운 동작을 유도

https://github.com/huggingface/lerobot/blob/main/src/lerobot/policies/act/modeling_act.py#L42

policy

ACTPolicy

ACTConfig

model

ACT

vae_encoder

ACTEncoder

ACTEncoderLayer

ACTTemporalEnsembler

encoder

ACTEncoder

ACTEncoderLayer

decoder

ACTDecoder

ACTDecoderLayer

ACTSinusoidalPositionEmbedding2d

04 Diffusion Policy 소개

Diffusion Policy :

Visuomotor Policy Learning via Action Diffusion

Denoising Diffusion Probabilistic Models(DDIM)

랜덤 노이즈를 놀라운 정도로 일관되고 사실적인 이미지로 변환하는 cutting-edge generative AI algorithm

- **Unconditional Generative Models**

- Generate images from random noise only
- Produced images can be anything, without any specific constraints or conditions.



외부 지침 없이 노이즈에서 이미지가 생성

- **Conditional Generative Models**

- Generate images from a combination of random noise and **input prompts**
- Prompt can take various forms, such as text descriptions, image or other types of data.

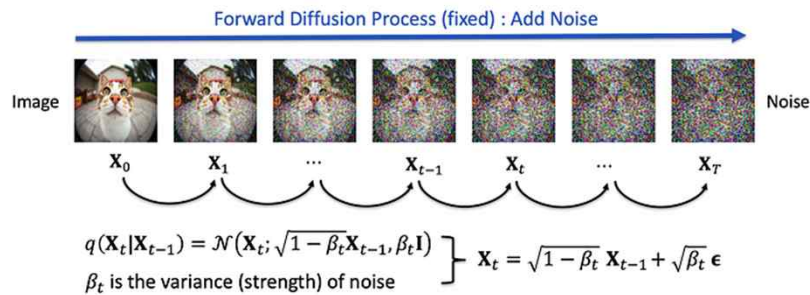


텍스트 프롬프트를 사용하여 이미지 생성 프로세스를 조정하여 출력을 정확하게 제어

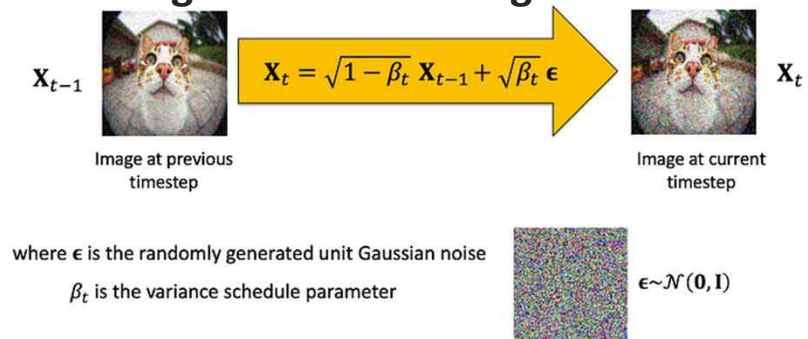
Forward Diffusion

Forward Diffusion은 이미지에 노이즈를 일련의 시간 단계에 걸쳐 추가하는 과정으로, 이미지가 완전히 노이즈가 될 때까지 진행

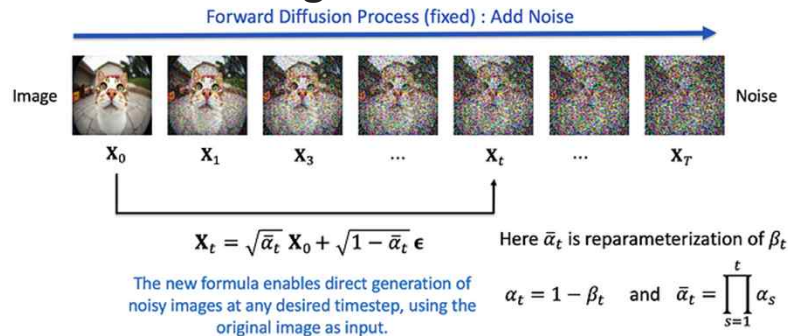
1 Forward Diffusion



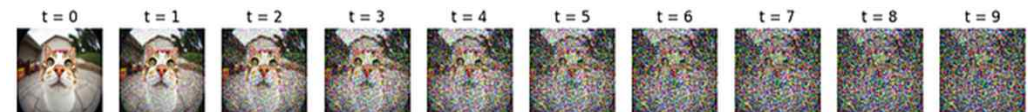
2 Adding Noise to an Image



3 Reformulating the Forward Diffusion



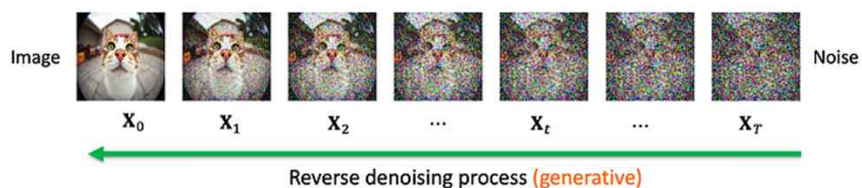
4 DDPM Scheduler



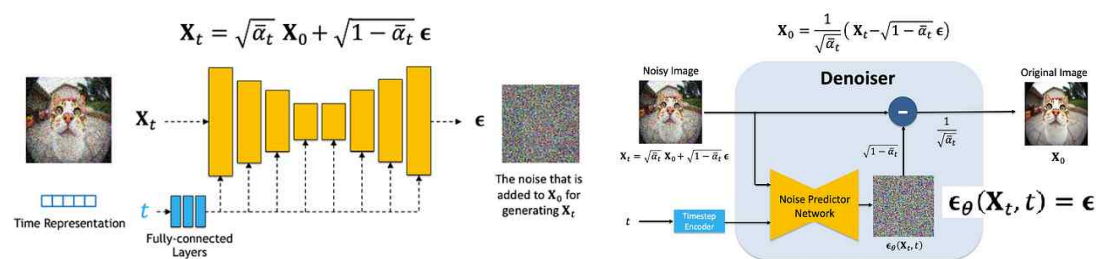
Reverse Diffusion

역 확산은 이미지에서 노이즈 제거 선명한 이미지 생성하는 프로세스. 순방향 확산의 반대

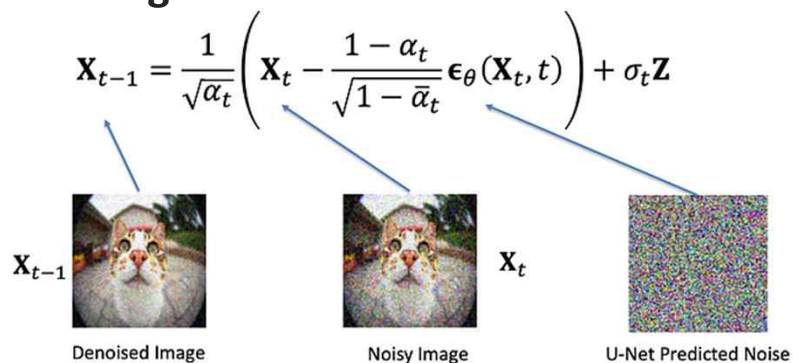
1 Reverse Diffusion



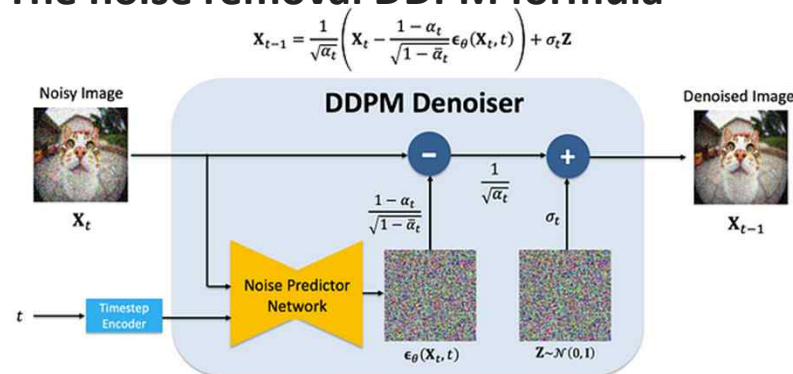
2 Training a Noise Predictor Neural Network for Reverse Diffusion



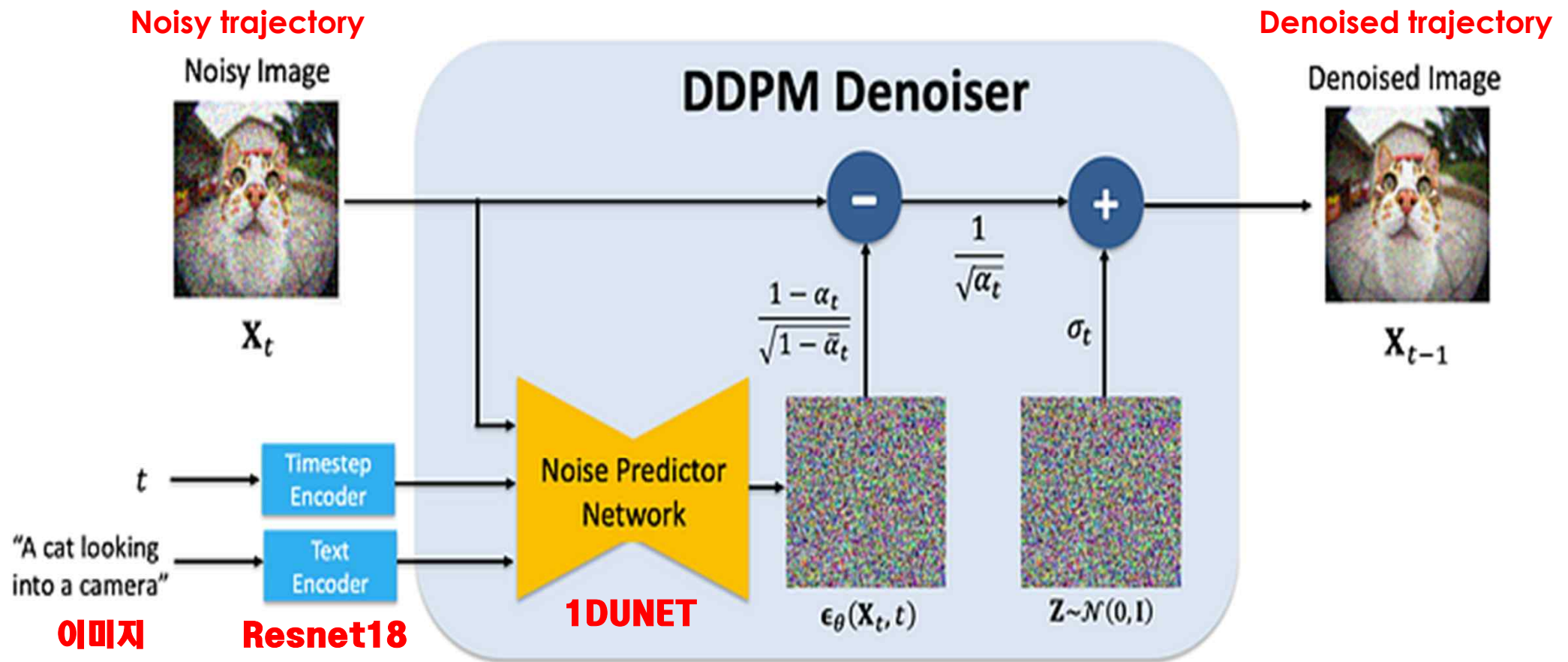
3 Scaling the Predicted Noise

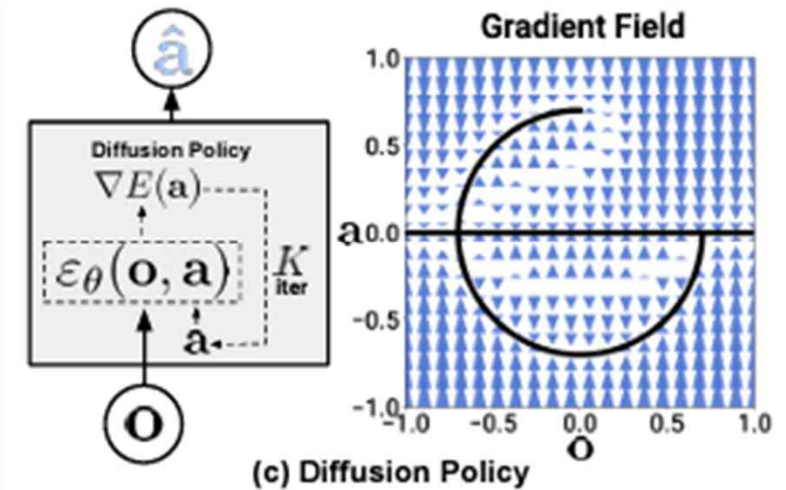
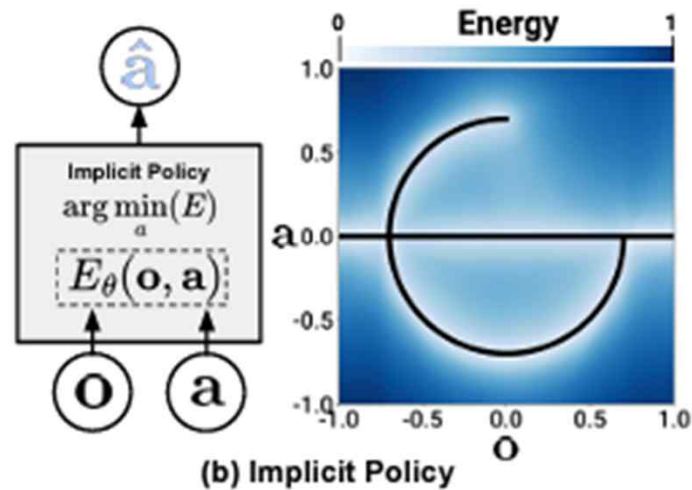
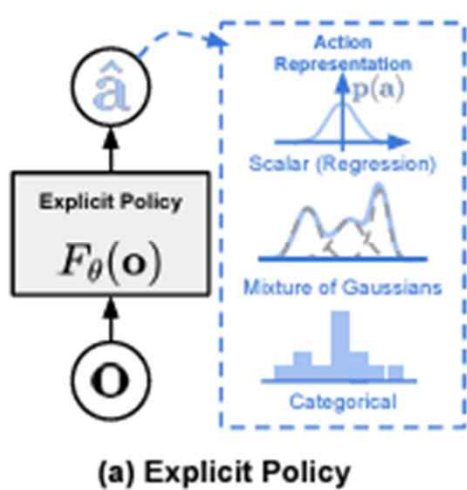


4 The noise removal DDPM formula



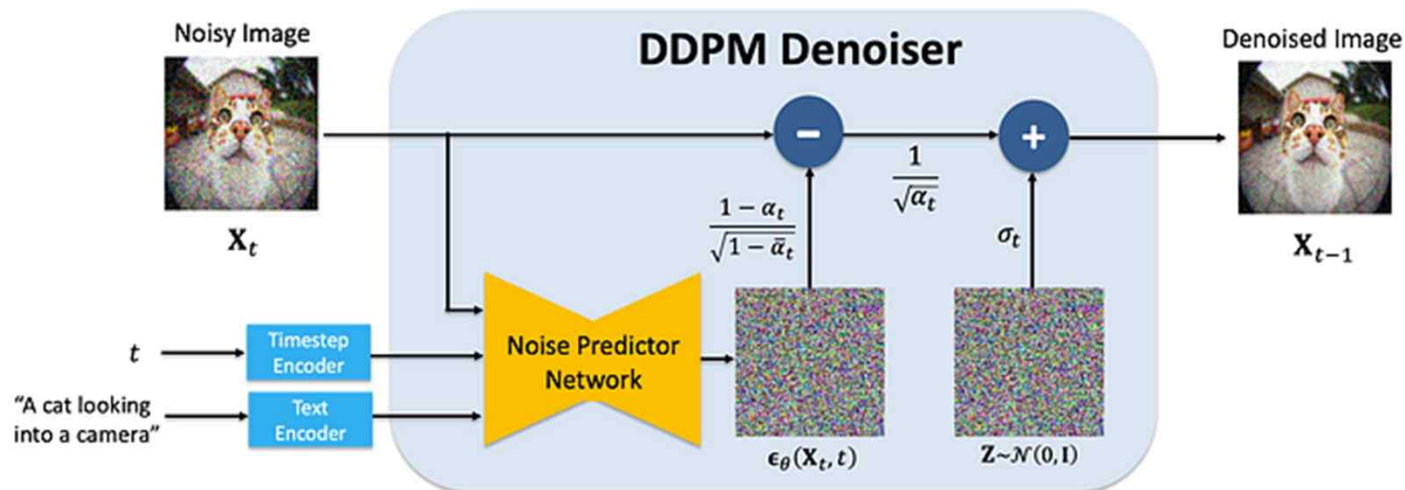
Diffusion Policy와 Conditional Diffusion Model의 관계



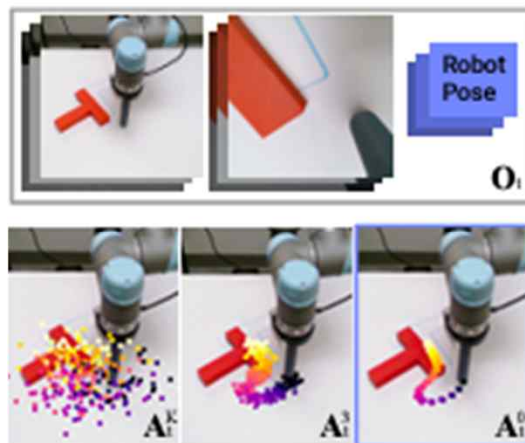


- a) 다양한 유형(**different types**)의 행동(**action**) 표현을 사용하는 명시적 정책(**Explicit policy**).
- b) 암시적 정책은 행동과 관찰 모두를 조건으로 하는 에너지 함수를 학습하고 에너지 환경을 최소화하는 행동(**action**) 최적화.
- c) 확산 정책은 학습된 그래디언트 필드를 통해 노이즈를 동작으로 구체화.
 - 이 공식은 안정적인 훈련을 제공하고, 학습된 정책이 다중 모드 동작 분포를 정확하게 모델링할 수 있도록 하며, 고차원 동작 시퀀스 수용함.

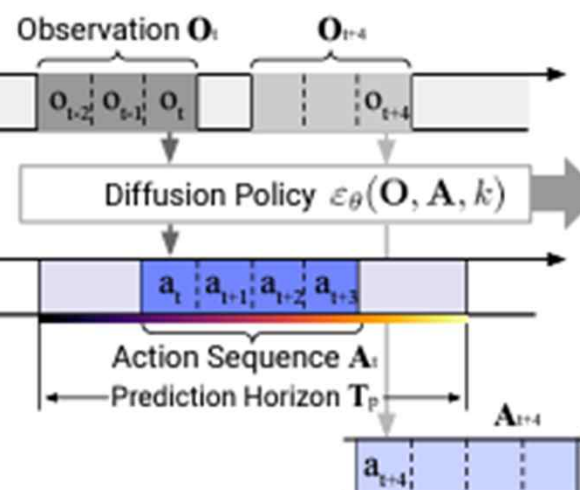
DDPM Training



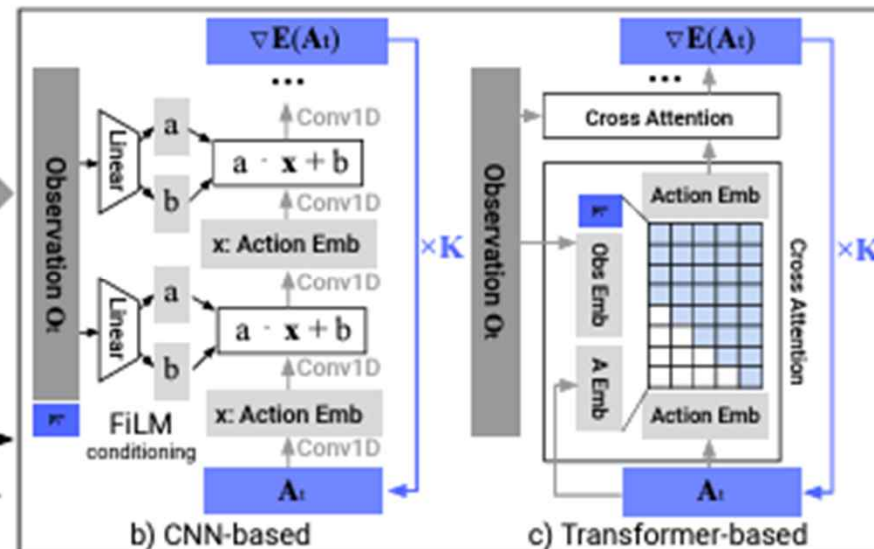
Input: Image Observation Sequence



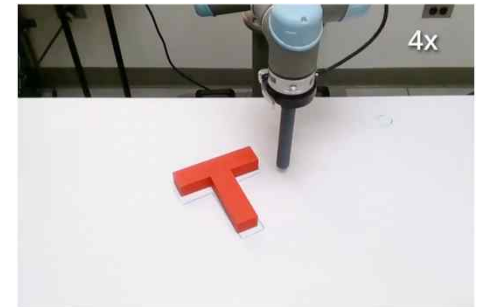
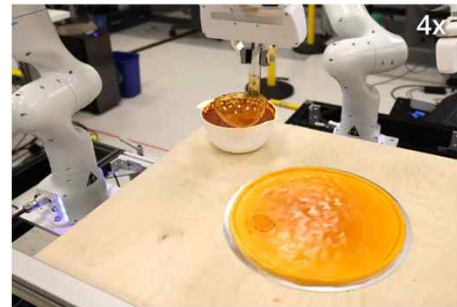
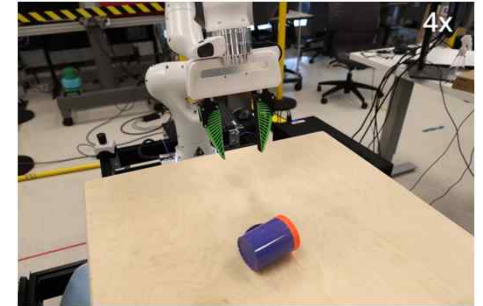
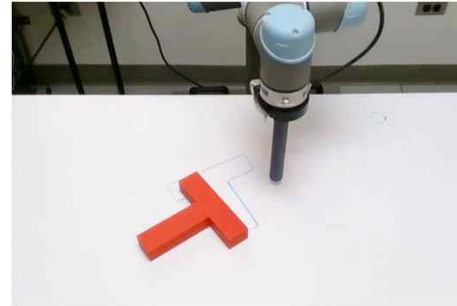
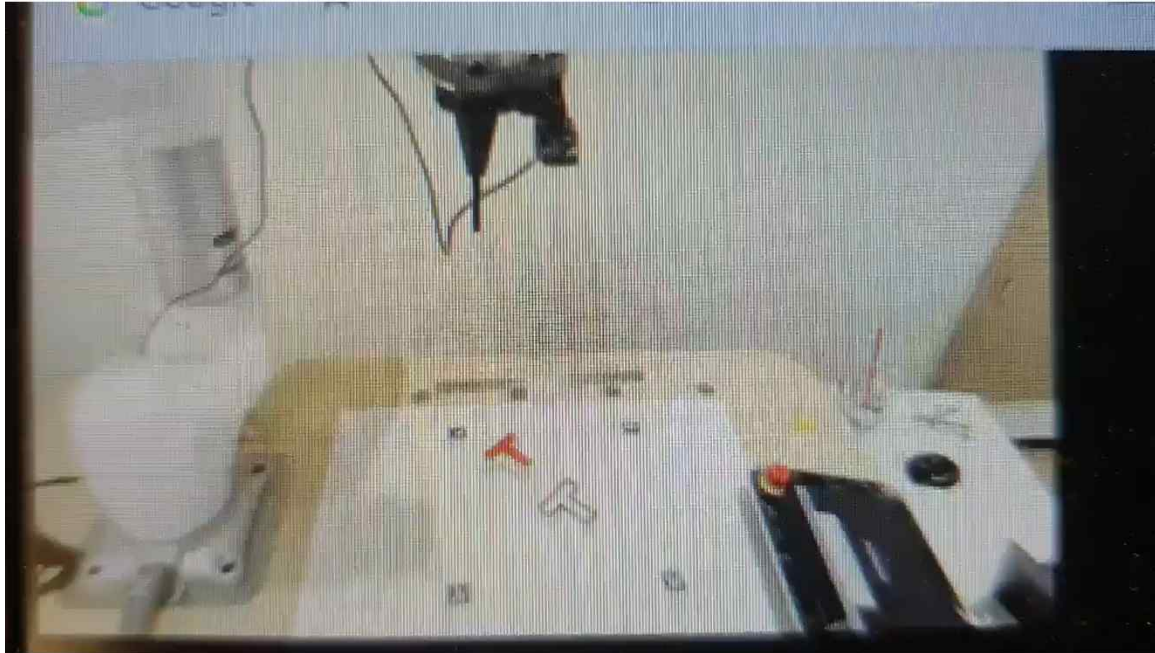
Output: Action Sequence



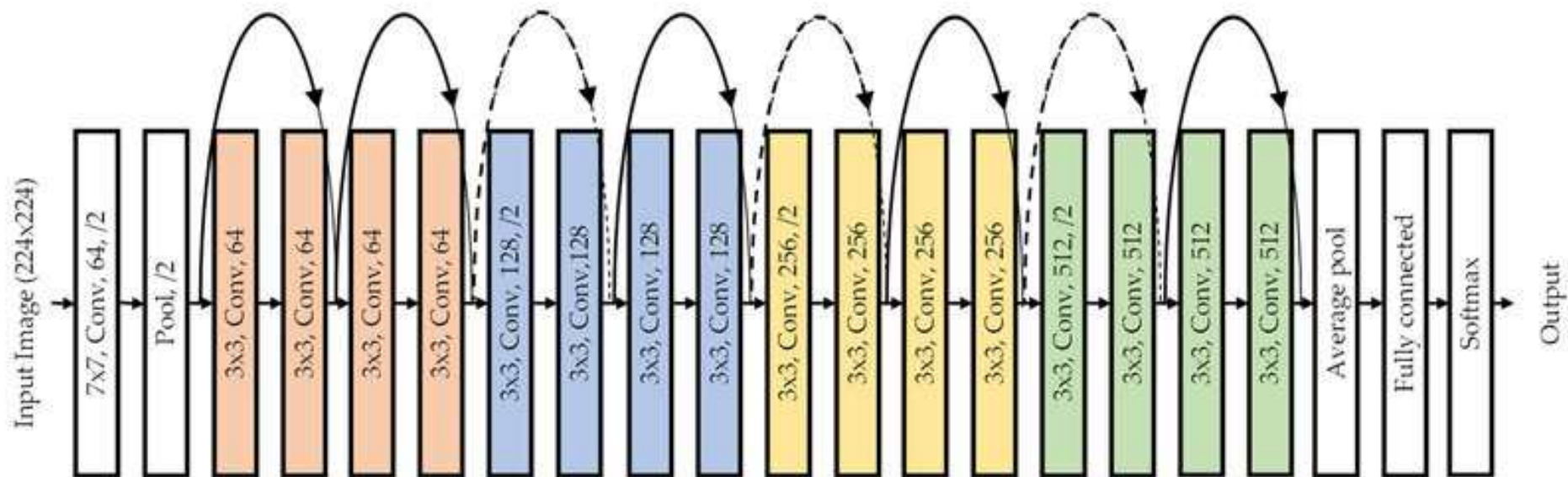
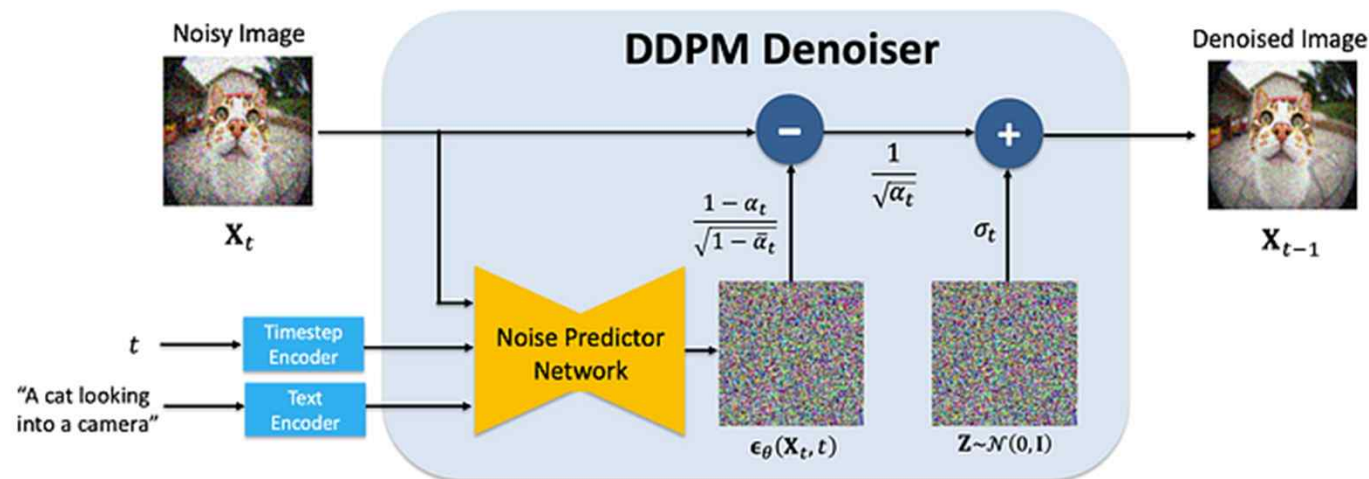
a) Diffusion Policy General Formulation



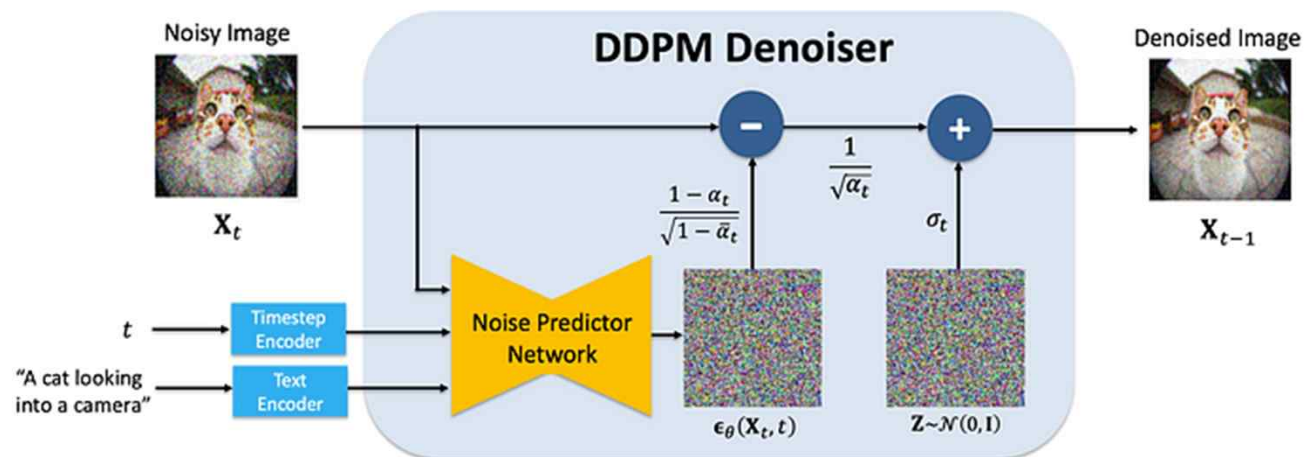
DDPM



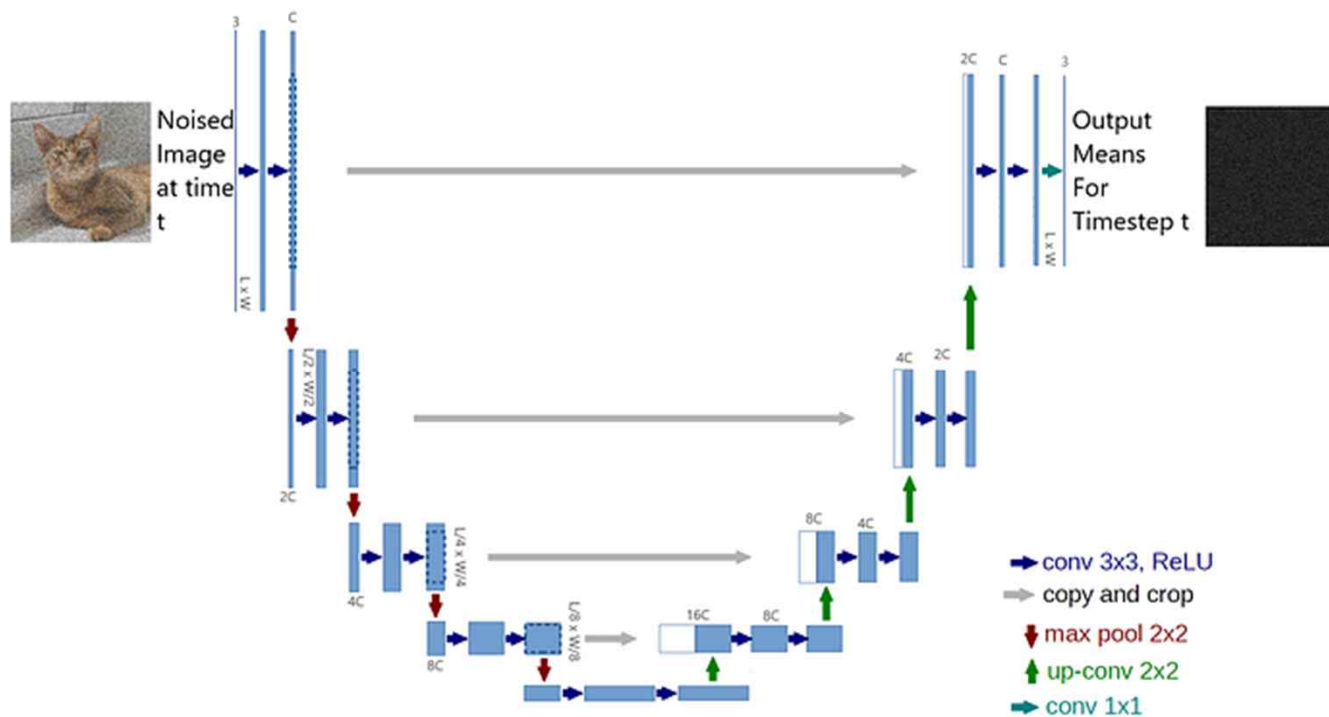
Vision Encoder



Conditional UNet1D



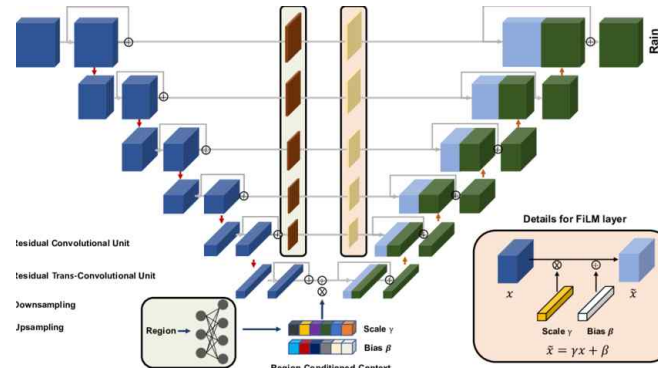
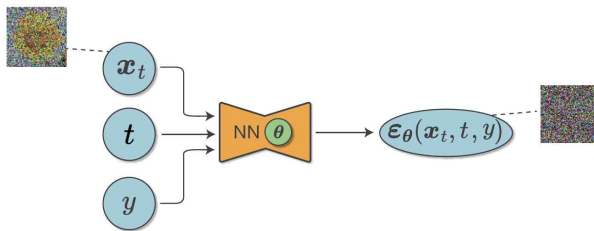
입력
이미지 대신
trajectory



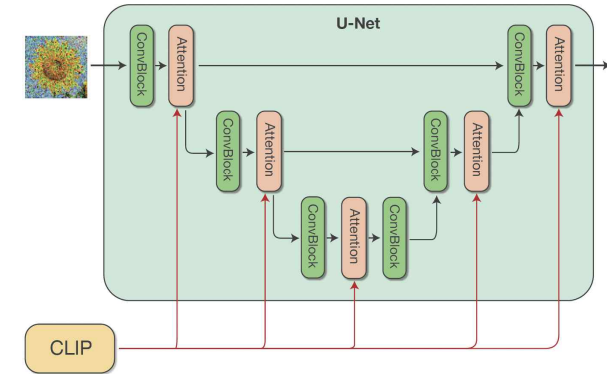
출력
이미지 대신
trajectory

조건 결합방법 비교

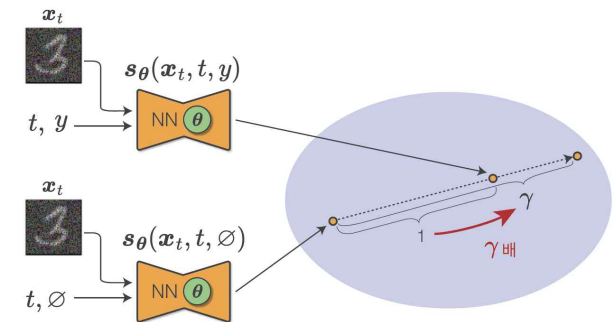
조건부 확산 모델의 신경망



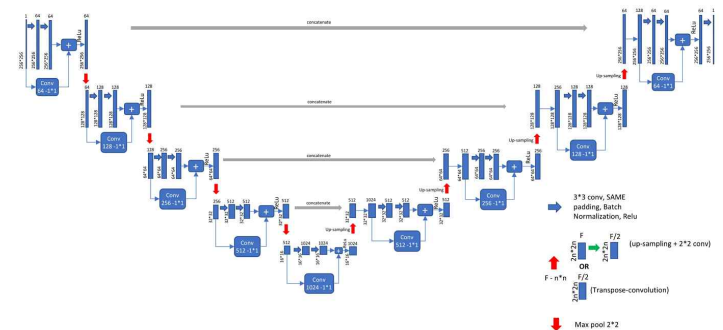
어텐션 계층을 도입한 U-Net의 예



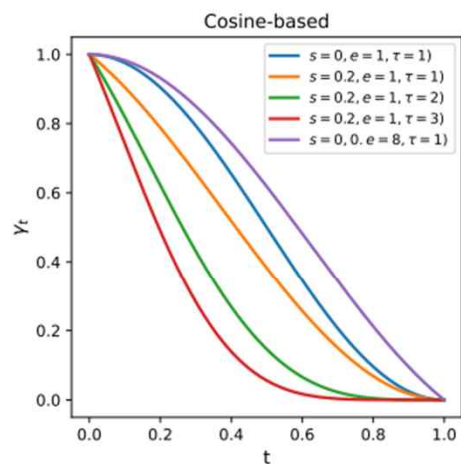
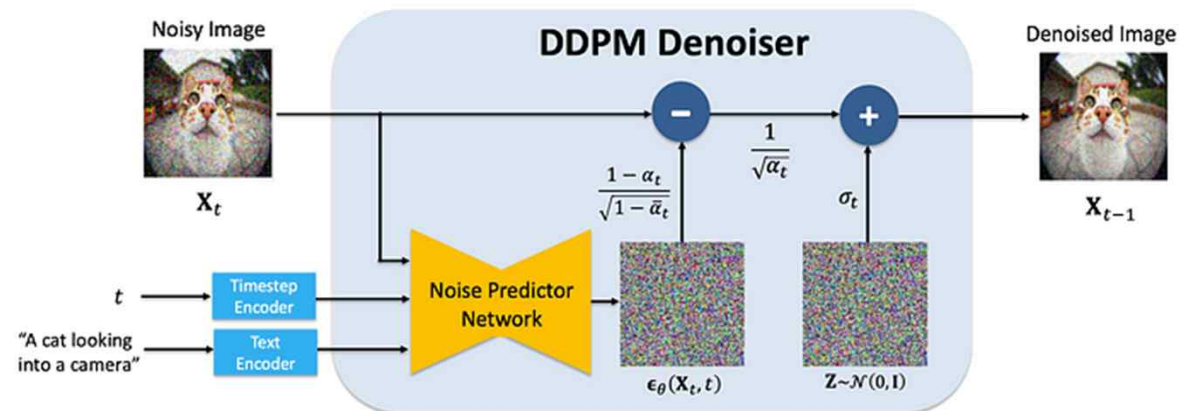
점수 추정 신경망을 이용한 분류기 없는 가이드선스



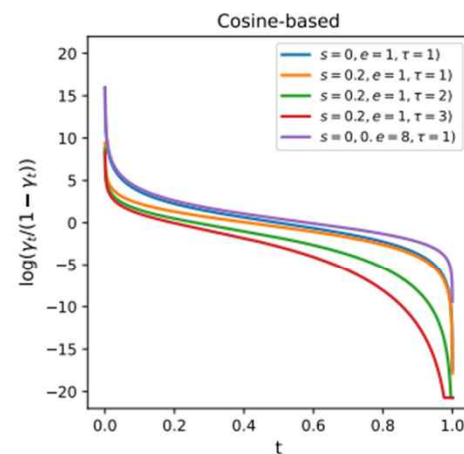
방식	수식 구조	조건 반영 위치	장점	단점
Concat	$[x, c]$	입력에서 결합	단순	조건 반영이 약할 수 있음
FiLM	$\gamma(c) \cdot x + \beta(c)$	Feature map에 직접 적용	연속적 제어	조건 표현력이 제한적
Cross-Attn	$\text{softmax}(QK^T)V$	UNet 또는 Transformer block	표현력 강력	연산량 증가
CFG	$(1 + w)\epsilon(x, c) - w\epsilon(x, \emptyset)$	샘플링 단계	조건 반영 강함	2회 forward 필요
Residual Mod	$x + f(x; c)$	residual block 내부	적응형 제어	구조 설계 필요



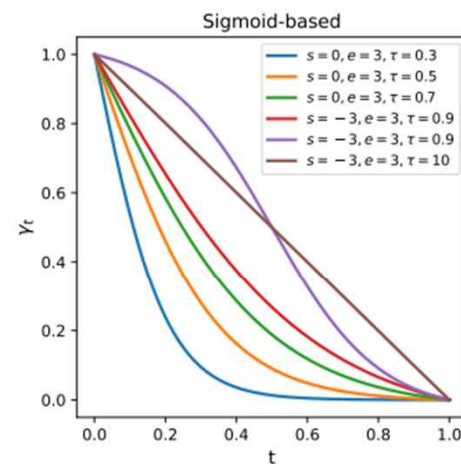
Noise Scheduler



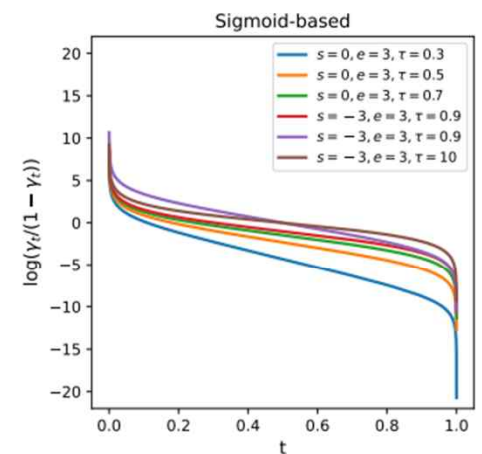
(a) Cosine



(b) Cosine (logSNR)

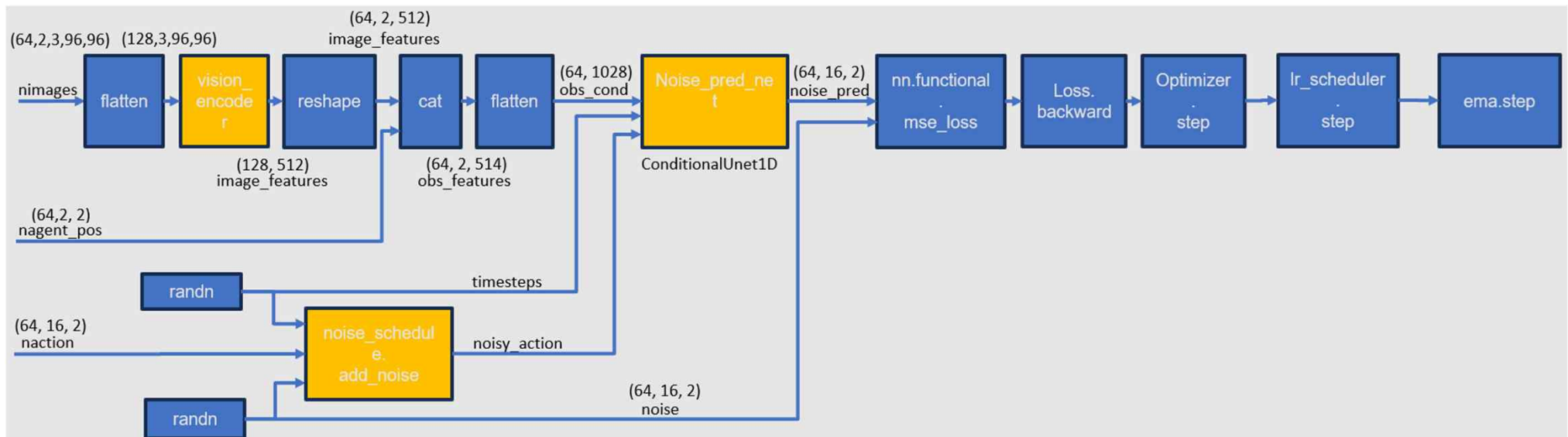
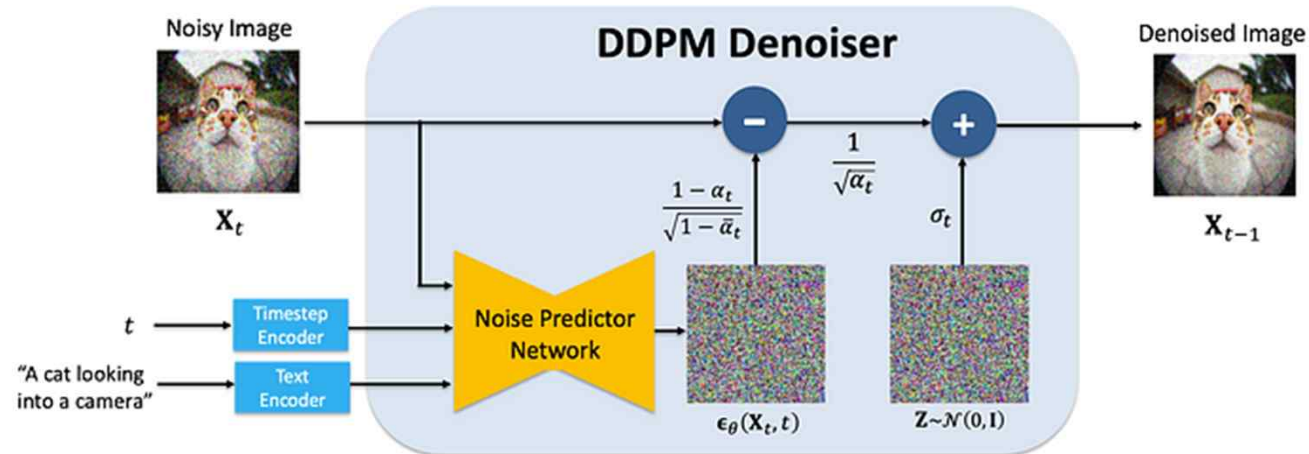


(c) Sigmoid

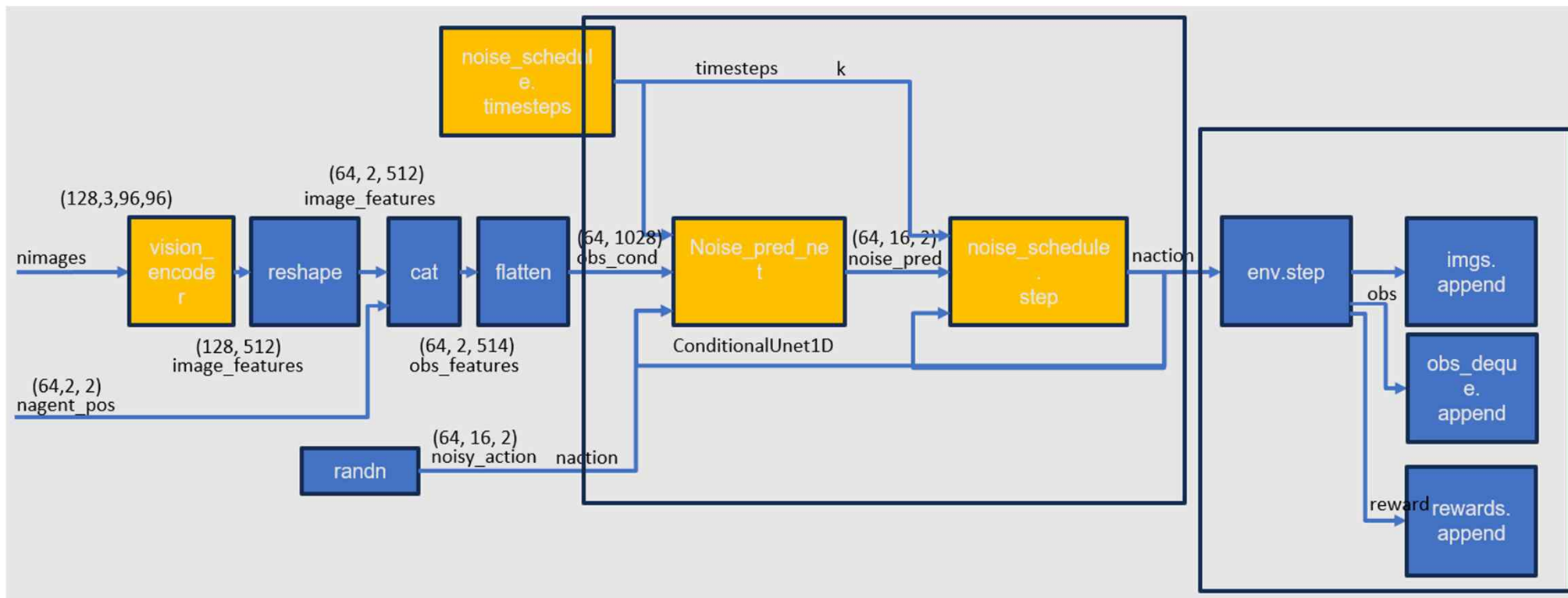


(d) Sigmoid (logSNR)

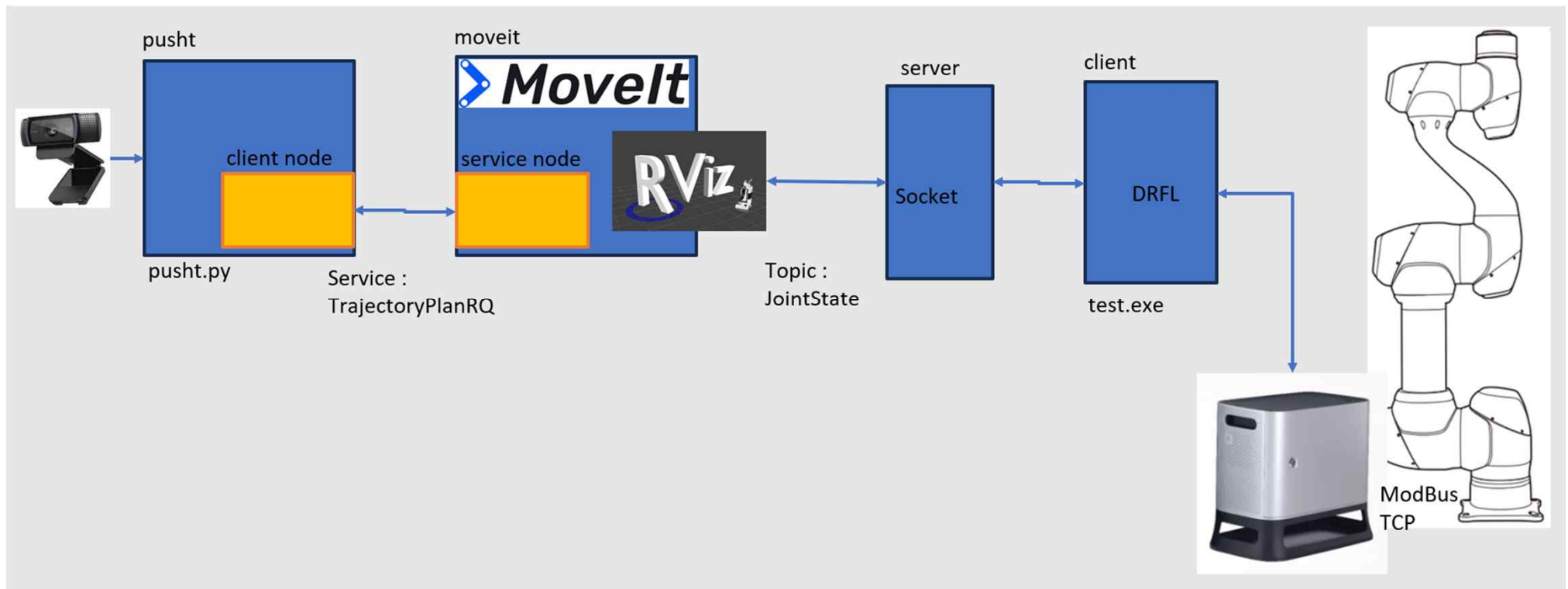
노이즈를 예측하도록 네트워크 학습

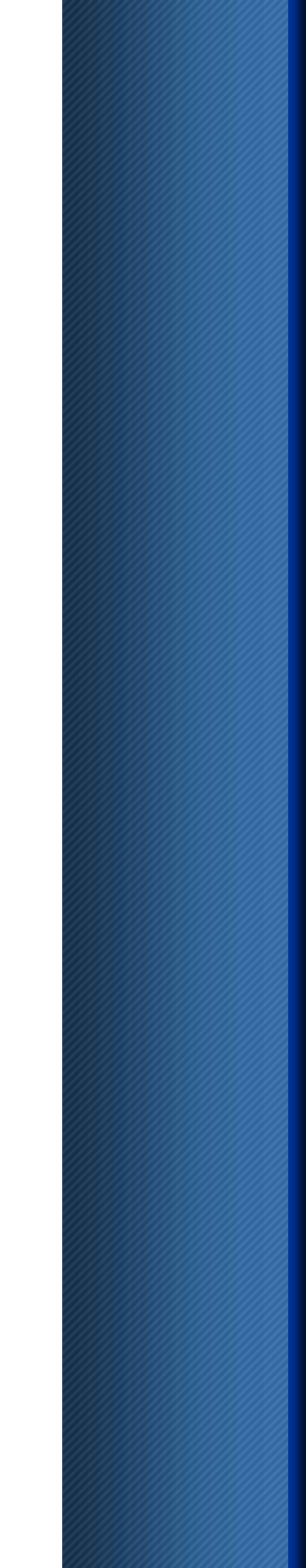


노이즈를 예측하고 노이즈 제거



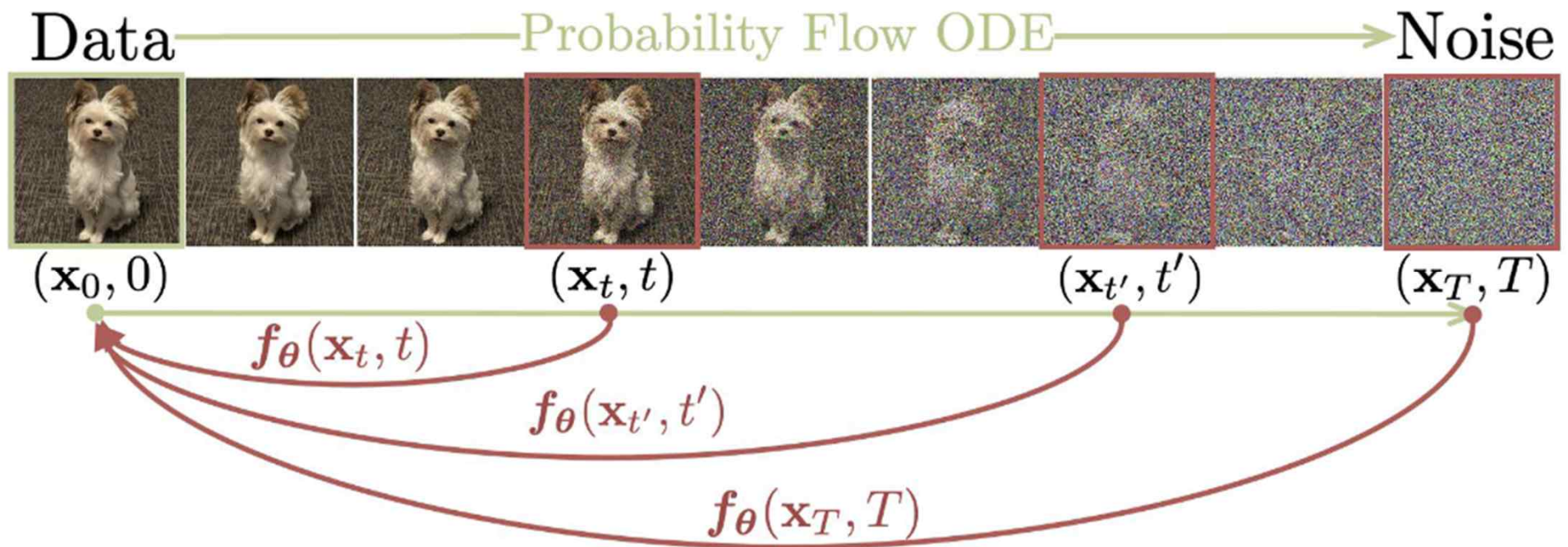
Diffusion Policy 구현



A solid blue vertical bar runs along the left edge of the slide.

04 Consistency Policy

Consistency Model



- Diffusion model의 느린 속도를 극복하는 Consistency Model이 제안 됨.
- Single step의 sampling 으로 고품질의 이미지 생성.

Consistency Model

- 임의의 (x_t, t) 쌍에 대해서 아래 식을 만족하도록 학습.
- $f(x_t, t) = f(x_{t'}, t')$ for all $t, t' \in [\epsilon, T]$

녹색 선: 여러 개의 ODE 궤적(trajecory) — 각기 다른 확률 흐름 (Probability Flow ODE)을 따라 데이터를 노이즈로 만드는 과정

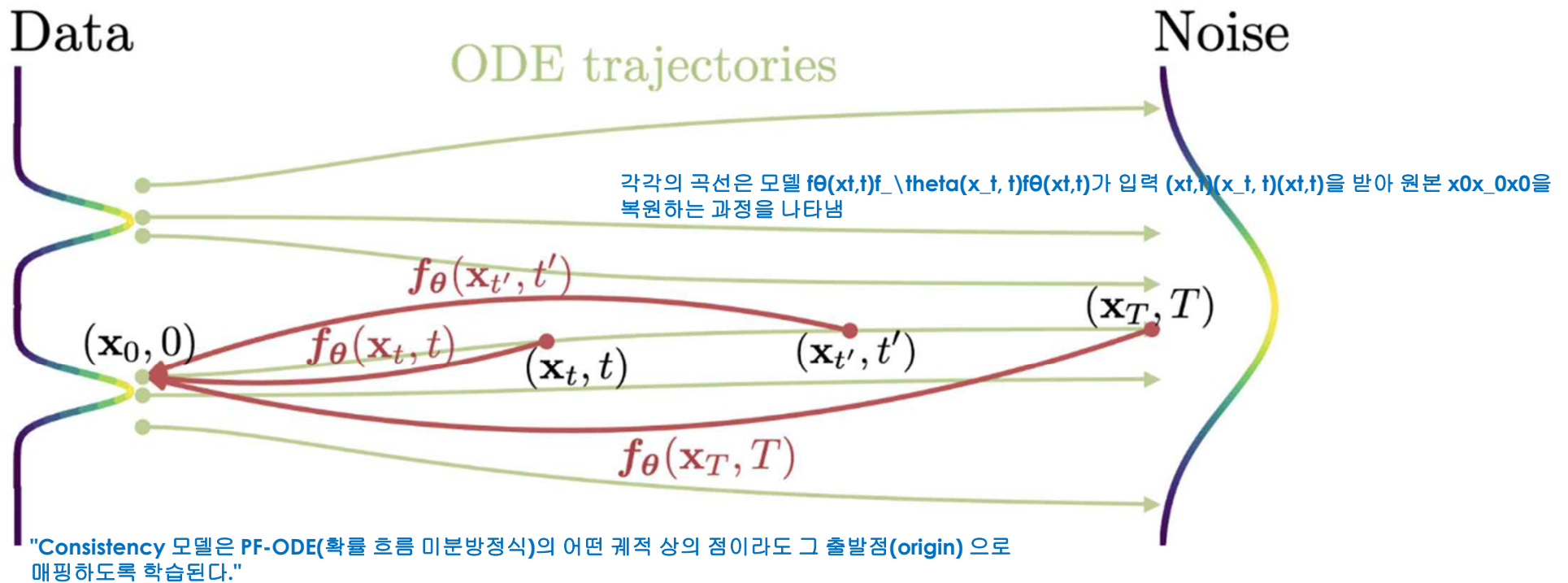


Figure 2: Consistency models are trained to map points on any trajectory of the PF ODE to the trajectory's origin.

Consistency Model



- Consistency Distillation
 - Pre-trained model 필요
- Isolation Method
 - Pre-trained model 필요하지 않음

Consistency Model

Parameterize self-consistency

$f(\mathbf{x}_t, t) = f(\mathbf{x}_{t'}, t')$ for all $t, t' \in [\epsilon, T]$.

$f(\mathbf{x}_\epsilon, \epsilon) = \mathbf{x}_\epsilon$. 즉, $f(\cdot, \epsilon)$ 은 identity function 이라 볼 수 있음. (Boundary condition).

$$f_\theta(\mathbf{x}, t) = \begin{cases} \mathbf{x} & t = \epsilon \\ F_\theta(\mathbf{x}, t) & t \in (\epsilon, T] \end{cases}.$$

Skip connection을 이용한 표현.

$$f_\theta(\mathbf{x}, t) = c_{\text{skip}}(t)\mathbf{x} + c_{\text{out}}(t)F_\theta(\mathbf{x}, t),$$

$c_{\text{skip}}(t)$ 와 $c_{\text{out}}(t)$ 은 미분가능한 함수이며, $c_{\text{skip}}(\epsilon) = 1$, $c_{\text{out}}(\epsilon) = 0$ 을 만족한다.

In EDM (Karras et al., 2022), authors choose

$$c_{\text{skip}}(t) = \frac{\sigma_{\text{data}}^2}{t^2 + \sigma_{\text{data}}^2}, \quad c_{\text{out}}(t) = \frac{\sigma_{\text{data}} t}{\sqrt{\sigma_{\text{data}}^2 + t^2}}, \quad \sigma_{\text{data}} = 0.5$$

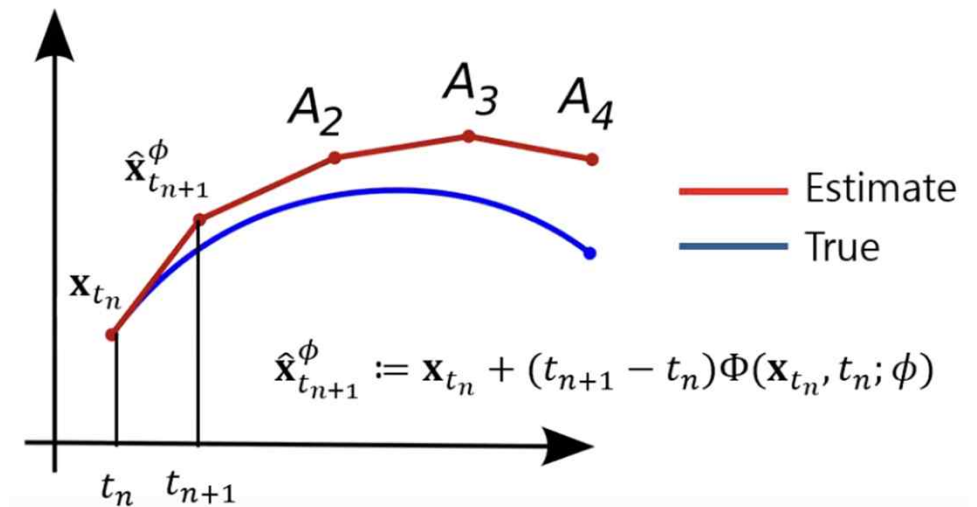
Consistency Model

Pre-trained score model $s_\phi(\mathbf{x}, t)$.

$$\text{ODE} \quad \frac{d\mathbf{x}_t}{dt} = -t s_\phi(\mathbf{x}_t, t).$$

Time $[\epsilon, T]$ 을 $N - 1$ 개의 sub-intervals로 discretizing.

$$t_1 = \epsilon < t_2 < \dots < t_N = T.$$



N 이 충분히 클 때, $\mathbf{x}_{t_{n+1}}$ 로 부터 $\mathbf{x}_{t_n}$ 을 numerical ODE solver를 통해 정확히 추정할 수 있음.

추정 값을 $\hat{\mathbf{x}}_{t_n}^\phi$ 이라고 하면, $\hat{\mathbf{x}}_{t_n}^\phi := \mathbf{x}_{t_{n+1}} + (t_n - t_{n+1})\Phi(\mathbf{x}_{t_{n+1}}, t_{n+1}; \phi)$, 라 표현 할 수 있음.

$\Phi(\dots; \phi)$ 는 ODE solver, 기울기.

Consistency Model

Algorithm 1 Multistep Consistency Sampling

Input: Consistency model $f_{\theta}(\cdot, \cdot)$, sequence of time points $\tau_1 > \tau_2 > \dots > \tau_{N-1}$, initial noise $\hat{\mathbf{x}}_T$

$\mathbf{x} \leftarrow f_{\theta}(\hat{\mathbf{x}}_T, T)$

for $n = 1$ **to** $N - 1$ **do**

 Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$\hat{\mathbf{x}}_{\tau_n} \leftarrow \mathbf{x} + \sqrt{\tau_n^2 - \epsilon^2} \mathbf{z}$

$\mathbf{x} \leftarrow f_{\theta}(\hat{\mathbf{x}}_{\tau_n}, \tau_n)$

end for

Output: $\mathbf{x}$

Consistency Model

Consistency distillation에서, $\nabla \log p_t(\mathbf{x})$ 를 근사하기 위해 pre-trained score model $s_\phi(\mathbf{x}, t)$ 를 이용했는데, 대신 아래와 같은 식을 이용한다.

$$\nabla \log p_t(\mathbf{x}_t) = -\mathbb{E} \left[\frac{\mathbf{x}_t - \mathbf{x}}{t^2} \mid \mathbf{x}_t \right],$$

where $\mathbf{x} \sim p_{\text{data}}$ and $\mathbf{x}_t \sim \mathcal{N}(\mathbf{x}; t^2 \mathbf{I})$. That is, given $\mathbf{x}$ and $\mathbf{x}_t$, we can estimate $\nabla \log p_t(\mathbf{x}_t)$ with $-(\mathbf{x}_t - \mathbf{x})/t^2$.

Consistency Model

Algorithm 3 Consistency Training (CT)

Input: dataset $\mathcal{D}$, initial model parameter θ , learning rate η , step schedule $N(\cdot)$, EMA decay rate schedule $\mu(\cdot)$, $d(\cdot, \cdot)$, and $\lambda(\cdot)$

$\theta^- \leftarrow \theta$ and $k \leftarrow 0$

repeat

 Sample $\mathbf{x} \sim \mathcal{D}$, and $n \sim \mathcal{U}[[1, N(k) - 1]]$

 Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$\mathcal{L}(\theta, \theta^-) \leftarrow$

$\lambda(t_n)d(\mathbf{f}_\theta(\mathbf{x} + t_{n+1}\mathbf{z}, t_{n+1}), \mathbf{f}_{\theta^-}(\mathbf{x} + t_n\mathbf{z}, t_n))$

$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta, \theta^-)$

$\theta^- \leftarrow \text{stopgrad}(\mu(k)\theta^- + (1 - \mu(k))\theta)$

$k \leftarrow k + 1$

until convergence

VLA와 확산형 모델

VLA

Plan

Task

Action/Trajectory

확산 모델

Trajectory



감사합니다.
